Starknet 시퀀서 구현체 분석

1. 개요

이 프로젝트는 **Starknet 네트워크의 시퀀서(Sequencer)**를 구현한 Rust 기반의 시스템입니다. 시퀀서는 Starknet L2 블록체인에서 트랜잭션을 수집, 정렬, 실행하고 블록을 생성하는 핵심 컴포넌트입니다.

1.1 Starknet에서의 시퀀서 역할

시퀀서는 Starknet L2 네트워크의 핵심 구성 요소로서 다음과 같은 중요한 역할을 수행합니다:

1.1.1 트랜잭션 라이프사이클 관리

- **트랜잭션 수집**: 사용자와 dApp으로부터 제출된 트랜잭션을 mempool에서 수집
- **유효성 검증**: 트랜잭션의 서명, nonce, 가스 한도 등을 검증
- **트랜잭션 정렬**: 수집된 트랜잭션을 우선순위와 nonce 순서에 따라 정렬
- **실행 및 상태 업데이트**: Cairo VM을 통해 트랜잭션을 실행하고 상태 변경사항 적용

1.1.2 블록 생성 및 관리

- **블록 제안**: 검증된 트랜잭션들을 묶어 새로운 블록 제안
- **블록 실행**: Blockifier를 통해 블록 내 모든 트랜잭션 실행
- **상태 루트 계산**: Patricia Tree를 사용하여 새로운 상태 루트 계산
- **블록 커밋**: 실행 완료된 블록을 체인에 커밋

1.1.3 네트워크 합의 및 동기화

- **합의 참여**: Tendermint 기반 합의 알고리즘에 참여
- **P2P 통신**: 다른 노드들과 블록 및 트랜잭션 정보 공유
- **상태 동기화**: 네트워크의 최신 상태와 동기화 유지

1.1.4 L1-L2 브릿지 관리

- **L1 연동**: Ethereum L1과의 상호작용을 통한 최종성 보장
- **상태 루트 제출**: 주기적으로 L2 상태 루트를 L1에 제출
- **L1↔L2 메시지 처리**: 크로스 레이어 메시지 전달 및 처리

2. 전체 아키텍처

2.1 모듈 구조

이 시퀀서는 **Apollo**라는 코드명으로 개발되었으며, 다음과 같은 주요 모듈들로 구성됩니다:

```
Apollo Sequencer
├── Core Components
|   ├── apollo_node (메인 노드)
|   ├── apollo_consensus (합의 메커니즘)
|   ├── apollo_mempool (트랜잭션 풀)
|   ├── apollo_gateway (API 게이트웨이)
|   └── apollo_batcher (배치 처리)
├── Execution Layer
|   ├── blockifier (블록 실행 엔진)
|   ├── starknet_api (Starknet API)
|   └── starknet_os (Starknet OS)
├── Storage & State
|   ├── apollo_storage (저장소)
|   ├── apollo_state_sync (상태 동기화)
|   └── starknet_patricia (패트리샤 트리)
└── L1 Integration
    ├── apollo_l1_provider (L1 연동)
    ├── apollo_l1_gas_price (가스 가격)
    └── papyrus_base_layer (기본 레이어)
```

3. 핵심 컴포넌트 분석

3.1 Apollo Node (apollo_node)

역할: 시퀀서의 메인 노드로서 모든 컴포넌트를 조율하고 관리하는 중앙 오케스트레이터

3.1.1 핵심 구조

```rust
pub struct SequencerNodeComponents {
    pub batcher: Option<Batcher>,                      // 블록 배치 처리
    pub class_manager: Option<ClassManager>,           // 컨트랙트 클래스 관리
    pub consensus_manager: Option<ConsensusManager>, // 합의 메커니즘
    pub gateway: Option<Gateway>,                      // API 게이트웨이
    pub http_server: Option<HttpServer>,              // HTTP 서버
    pub l1_endpoint_monitor: Option<L1EndpointMonitor>, // L1 엔드포인트 모니터링
    pub l1_scraper: Option<L1Scraper>,                // L1 데이터 스크래핑
    pub l1_provider: Option<L1Provider>,              // L1 연동 서비스
    pub mempool: Option<MempoolCommunicationWrapper>, // 트랜잭션 풀
    pub monitoring_endpoint: Option<MonitoringEndpoint>, // 모니터링
    pub state_sync: Option<StateSync>,                // 상태 동기화
    pub sierra_compiler: Option<SierraCompiler>,      // Sierra 컴파일러
    pub signature_manager: Option<SignatureManager>, // 서명 관리
    // ... 기타 컴포넌트들
}
```

3.1.2 컴포넌트 생성 및 초기화 과정

1단계: 설정 검증 및 로드

```rust
// main.rs에서 실행되는 초기화 과정
let config = load_and_validate_config(args().collect())
    .expect("Failed to load and validate config");
```

2단계: 컴포넌트별 실행 모드 결정

각 컴포넌트는 다음 실행 모드 중 하나로 설정됩니다:

- `LocalExecutionWithRemoteDisabled` : 로컬에서만 실행
- `LocalExecutionWithRemoteEnabled` : 로컬 실행 + 원격 백업
- `Remote` : 원격 서비스 사용
- `Disabled` : 비활성화

3단계: 의존성 기반 컴포넌트 생성

```rust
pub async fn create_node_components(
    config: &SequencerNodeConfig,
    clients: &SequencerNodeClients,
) -> SequencerNodeComponents {
    // 각 컴포넌트를 의존성 순서에 따라 생성
    // 예: L1 Scraper -> L1 Provider -> Batcher 순서
}
```

3.1.3 주요 기능

컴포넌트 라이프사이클 관리:

- 컴포넌트 생성 및 초기화
- 의존성 주입 및 클라이언트 연결
- 컴포넌트 간 통신 채널 설정
- 장애 발생 시 복구 메커니즘

설정 관리:

- TOML 기반 설정 파일 파싱
- 환경별 설정 오버라이드
- 런타임 설정 변경 지원
- 설정 유효성 검증

서버 실행 및 관리:

- 비동기 서버 실행
- 그레이스풀 셧다운 처리
- 패닉 핸들링 및 프로세스 종료
- 메모리 할당자 최적화 (JeMalloc 사용)

3.2 Consensus Manager (apollo_consensus)

역할: Starknet 네트워크의 합의 메커니즘을 구현하는 핵심 컴포넌트

3.2.1 Tendermint 기반 합의 알고리즘

합의 프로토콜 구조:

```
/// Tendermint 기반 합의 구현
/// - Observer 모드: 합의 메시지 수신 및 결정 업데이트
/// - Active 모드: 메시지 송수신 및 능동적 합의 참여
pub struct ConsensusManager {
    config: ConsensusManagerConfig,
    batcher_client: SharedBatcherClient,
    state_sync_client: SharedStateSyncClient,
    class_manager_client: SharedClassManagerClient,
    signature_manager_client: SharedSignatureManagerClient,
```

```
    l1_gas_price_client: SharedL1GasPriceClient,
}
```

3.2.2 합의 과정 상세 분석

1단계: 제안 단계 (Propose Phase)

- 리더 노드가 새로운 블록 제안
- 제안에는 트랜잭션 목록과 상태 변경사항 포함
- 제안 검증 및 네트워크 전파

2단계: 사전 투표 (Prevote Phase)

- 각 검증자가 제안에 대한 사전 투표 실시
- 2/3 이상의 사전 투표 수집 시 다음 단계 진행
- 타임아웃 메커니즘으로 무한 대기 방지

3단계: 사전 커밋 (Precommit Phase)

- 사전 투표 결과를 바탕으로 사전 커밋 투표
- 2/3 이상의 사전 커밋 수집 시 블록 확정
- 비잔틴 장애 허용 (BFT) 보장

4단계: 커밋 (Commit Phase)

- 확정된 블록을 체인에 커밋
- 상태 업데이트 및 다음 높이로 진행
- 최종성 보장

3.2.3 네트워크 통신 메커니즘

P2P 메시지 타입:

- **Proposal Messages**: 블록 제안 메시지
- **Vote Messages**: 투표 메시지 (Prevote, Precommit)
- **Timeout Messages**: 타임아웃 처리 메시지

메시지 검증 과정:

```
// 합의 메시지 검증 로직
fn validate_consensus_message(message: &ConsensusMessage) -> Result<(), Consensu
    // 1. 서명 검증
```

```
    // 2. 높이 및 라운드 검증
    // 3. 제안자 권한 검증
    // 4. 메시지 형식 검증
}
```

3.2.4 상태 머신 구현

합의 상태 전이:

- `NewHeight` : 새로운 높이 시작
- `Propose` : 제안 대기 또는 생성
- `Prevote` : 사전 투표 수집
- `Precommit` : 사전 커밋 수집
- `Commit` : 블록 커밋 및 완료

타임아웃 관리:

- 각 단계별 타임아웃 설정
- 네트워크 지연 고려한 동적 타임아웃
- 타임아웃 발생 시 다음 라운드 진행

3.2.5 성능 최적화 특징

비동기 처리:

- Tokio 기반 비동기 런타임 사용
- 동시 메시지 처리 및 검증
- 논블로킹 I/O 작업

메모리 효율성:

- LRU 캐시를 통한 메시지 캐싱
- 불필요한 메시지 정리
- 메모리 풀 활용

네트워크 최적화:

- 메시지 압축 및 배치 전송
- 연결 풀링 및 재사용
- 대역폭 사용량 모니터링

3.3 Mempool (apollo_mempool)

역할: 트랜잭션 메모리 풀을 관리하여 효율적인 트랜잭션 처리 및 블록 생성을 지원

3.3.1 Mempool 핵심 구조

```rust
pub struct Mempool {
    config: MempoolConfig,
    // Declare 트랜잭션 지연 처리를 위한 큐
    delayed_declares: AddTransactionQueue,
    // 모든 트랜잭션을 저장하는 풀
    tx_pool: TransactionPool,
    // 시퀀싱 가능한 트랜잭션들의 우선순위 큐
    tx_queue: TransactionQueue,
    // nonce 갭이 있는 계정들 (제거 후보)
    accounts_with_gap: AccountsWithGap,
    // Mempool 상태 관리
    state: MempoolState,
    clock: Arc<dyn Clock>,
}
```

3.3.2 트랜잭션 상태 관리 시스템

MempoolState 구조:

```rust
pub struct MempoolState {
    // 최근 커밋 이벤트 히스토리 (롤백 지원)
    commit_history: CommitHistory,
    // 블록에 커밋된 최종 nonce들
    committed: AddressToNonce,
    // 블록 생성 중 임시로 증가된 nonce들
    staged: AddressToNonce,
}
```

Nonce 해결 로직:

```rust
fn resolve_nonce(&self, address: ContractAddress, incoming_account_nonce: Nonce)
    // 1. Staged nonce 확인 (블록 생성 중)
    // 2. Committed nonce 확인 (최종 커밋됨)
    // 3. 둘 다 없으면 incoming nonce 사용
    self.staged
        .get(&address)
        .or_else(|| self.committed.get(&address))
```

```
        .copied()
        .unwrap_or(incoming_account_nonce)
}
```

3.3.3 트랜잭션 추가 과정

1단계: 기본 검증

```
pub fn add_tx(&mut self, args: AddTransactionArgs) -> MempoolResult<()> {
    // 1. 만료된 트랜잭션 제거
    let mut account_nonce_updates = self.remove_expired_txs();

    // 2. 지연된 Declare 트랜잭션 처리
    self.add_ready_declares();

    // 3. 트랜잭션 참조 생성 및 검증
    let tx_reference = TransactionReference::new(&args.tx);
    self.validate_incoming_tx(tx_reference, args.account_state.nonce)?;
}
```

2단계: 수수료 에스컬레이션 처리

```
fn handle_fee_escalation(&mut self, incoming_tx: &InternalRpcTransaction) -> Mem
    // 동일한 nonce의 기존 트랜잭션이 있는지 확인
    // 수수료가 충분히 증가했는지 검증
    // 기존 트랜잭션 제거 및 새 트랜잭션 추가
}
```

3단계: 용량 관리

```
fn handle_capacity_overflow(&mut self, tx: &InternalRpcTransaction, account_nonc
    // 메모리 한도 초과 시 제거 가능한 계정 선택
    // nonce 갭이 있는 계정들을 우선적으로 제거
    // 공간 확보 후 새 트랜잭션 추가
}
```

3.3.4 트랜잭션 선택 및 정렬

우선순위 기반 선택:

```rust
pub fn get_txs(&mut self, n_txs: usize) -> MempoolResult<Vec<InternalRpcTransact
    let mut eligible_tx_references: Vec<TransactionReference> = Vec::with_capaci

    while n_remaining_txs > 0 && self.tx_queue.has_ready_txs() {
        // 1. 준비된 트랜잭션 청크 가져오기
        let chunk = self.tx_queue.pop_ready_chunk(n_remaining_txs);

        // 2. 만료된 트랜잭션 제거
        let (valid_txs, expired_txs_updates) = self.prune_expired_nonqueued_txs(

        // 3. 다음 적격 트랜잭션들을 큐에 추가
        self.enqueue_next_eligible_txs(&valid_txs)?;

        // 4. 선택된 트랜잭션들의 nonce를 staged 상태로 변경
        for tx_reference in &valid_txs {
            self.state.stage(tx_reference)?;
        }
    }
}
```

3.3.5 블록 커밋 처리

커밋 과정:

```rust
pub fn commit_block(&mut self, args: CommitBlockArgs) {
    let CommitBlockArgs { address_to_nonce, rejected_tx_hashes } = args;

    // 1. 커밋된 nonce들로 mempool 상태 정렬
    for (&address, &next_nonce) in &address_to_nonce {
        // nonce 검증 및 업데이트
        // 오래된 트랜잭션들 제거
        // nonce 갭 해결
    }

    // 2. 거부된 트랜잭션들 제거
    for tx_hash in rejected_tx_hashes {
        if let Ok(tx) = self.tx_pool.remove(tx_hash) {
            self.tx_queue.remove(tx.contract_address());
        }
    }

    // 3. 상태 커밋 및 되감기 처리
```

```
    let addresses_to_rewind = self.state.commit(address_to_nonce);
}
```

3.3.6 특수 트랜잭션 처리

Declare 트랜잭션 지연 처리:

```
// Declare 트랜잭션은 컴파일 시간을 고려하여 지연 처리
if let InternalRpcTransactionWithoutTxHash::Declare(_) = &args.tx.tx {
    self.delayed_declares.push_back(self.clock.now(), args);
} else {
    self.add_tx_inner(args);
}
```

3.3.7 메트릭스 및 모니터링

수집되는 메트릭스:

- `MEMPOOL_POOL_SIZE` : 전체 트랜잭션 풀 크기
- `MEMPOOL_PRIORITY_QUEUE_SIZE` : 우선순위 큐 크기
- `MEMPOOL_PENDING_QUEUE_SIZE` : 대기 중인 트랜잭션 수
- `MEMPOOL_DELAYED_DECLARES_SIZE` : 지연된 Declare 트랜잭션 수
- `MEMPOOL_EVICTIONS_COUNT` : 제거된 트랜잭션 수
- `MEMPOOL_TOTAL_SIZE_BYTES` : 총 메모리 사용량

3.3.8 P2P 전파 메커니즘

트랜잭션 전파 과정:

1. 새 트랜잭션이 mempool에 추가됨
2. P2P propagator가 트랜잭션을 네트워크에 전파
3. 다른 노드들이 트랜잭션을 수신하고 검증
4. 중복 제거 및 우선순위 기반 전파 제어

3.4 Gateway (apollo_gateway)

역할: 외부 클라이언트와 시퀀서 간의 API 인터페이스를 제공하는 진입점

3.4.1 Gateway 핵심 구조

```rust
#[derive(Clone)]
pub struct Gateway {
    pub config: Arc<GatewayConfig>,
    // 상태 없는 트랜잭션 검증기
    pub stateless_tx_validator: Arc<StatelessTransactionValidator>,
    // 상태 기반 트랜잭션 검증기
    pub stateful_tx_validator: Arc<StatefulTransactionValidator>,
    // 상태 읽기 팩토리
    pub state_reader_factory: Arc<dyn StateReaderFactory>,
    // Mempool 클라이언트
    pub mempool_client: SharedMempoolClient,
    // 트랜잭션 변환기
    pub transaction_converter: Arc<TransactionConverter>,
    // 체인 정보
    pub chain_info: Arc<ChainInfo>,
}
```

3.4.2 트랜잭션 처리 파이프라인

전체 처리 과정:

```rust
pub async fn add_tx(
    &self,
    tx: RpcTransaction,
    p2p_message_metadata: Option<BroadcastedMessageMetadata>,
) -> GatewayResult<GatewayOutput> {
    // 1. Declare 권한 검사
    if let RpcTransaction::Declare(ref declare_tx) = tx {
        self.check_declare_permissions(declare_tx)?;
    }

    // 2. 메트릭 카운터 초기화
    let mut metric_counters = GatewayMetricHandle::new(&tx, &p2p_message_metadat
    metric_counters.count_transaction_received();

    // 3. CPU 집약적 작업을 별도 스레드에서 실행
    let blocking_task = ProcessTxBlockingTask::new(self, tx.clone(), tokio::runt
    let add_tx_args = tokio::task::spawn_blocking(move || blocking_task.process_

    // 4. Gateway 출력 생성
    let gateway_output = create_gateway_output(&add_tx_args.tx);

    // 5. Mempool에 트랜잭션 전송
    let add_tx_args = AddTransactionArgsWrapper { args: add_tx_args, p2p_message
```

```
    mempool_client_result_to_deprecated_gw_result(
        self.mempool_client.add_tx(add_tx_args).await,
    )?;

    metric_counters.transaction_sent_to_mempool();
    Ok(gateway_output)
}
```

3.4.3 트랜잭션 검증 시스템

1단계: 상태 없는 검증 (Stateless Validation)

```
impl StatelessTransactionValidator {
    pub fn validate(&self, tx: &RpcTransaction) -> GatewayResult<()> {
        // 1. 트랜잭션 형식 검증
        // 2. 서명 형식 검증
        // 3. 필드 범위 검증
        // 4. 가스 한도 검증
        // 5. 버전 호환성 검증
    }
}
```

2단계: 상태 기반 검증 (Stateful Validation)

```
impl StatefulTransactionValidator {
    pub fn extract_state_nonce_and_run_validations(
        &self,
        executable_tx: &ExecutableTransaction,
        mempool_client: SharedMempoolClient,
        validator: AccountTransactionValidator,
        runtime: tokio::runtime::Handle,
    ) -> Result<Nonce, String> {
        // 1. 계정 nonce 추출
        // 2. 계정 잔액 검증
        // 3. 컨트랙트 존재 여부 확인
        // 4. 실행 가능성 검증
        // 5. 리소스 바운드 검증
    }
}
```

3.4.4 트랜잭션 변환 과정

RPC → Internal RPC 변환:

```rust
pub async fn convert_rpc_tx_to_internal_rpc_tx(
    &self,
    tx: RpcTransaction,
) -> Result<InternalRpcTransaction, TransactionConverterError> {
    match tx {
        RpcTransaction::Declare(declare_tx) => {
            // 1. 클래스 해시 계산
            // 2. 컴파일된 클래스 해시 검증
            // 3. Sierra → CASM 변환
            // 4. Internal Declare 트랜잭션 생성
        }
        RpcTransaction::DeployAccount(deploy_tx) => {
            // 1. 컨트랙트 주소 계산
            // 2. 생성자 호출 데이터 검증
            // 3. Internal DeployAccount 트랜잭션 생성
        }
        RpcTransaction::Invoke(invoke_tx) => {
            // 1. 호출 데이터 검증
            // 2. Internal Invoke 트랜잭션 생성
        }
    }
}
```

Internal RPC → Executable 변환:

```rust
pub async fn convert_internal_rpc_tx_to_executable_tx(
    &self,
    tx: InternalRpcTransaction,
) -> Result<ExecutableTransaction, TransactionConverterError> {
    // 1. 트랜잭션 타입별 실행 가능한 형태로 변환
    // 2. 필요한 클래스 정보 로드
    // 3. 실행 컨텍스트 설정
}
```

3.4.5 권한 관리 시스템

Declare 권한 검사:

```rust
fn check_declare_permissions(&self, declare_tx: &RpcDeclareTransaction) -> Resul
    // 1. Declare 트랜잭션 차단 여부 확인
    if self.config.block_declare {
        return Err(StarknetError::blocked_transaction_type());
    }
```

```
    // 2. 승인된 선언자인지 확인
    let RpcDeclareTransaction::V3(declare_v3_tx) = declare_tx;
    if !self.config.is_authorized_declarer(&declare_v3_tx.sender_address) {
        return Err(StarknetError::unauthorized_declare());
    }

    Ok(())
}
```

3.4.6 상태 읽기 시스템

SyncStateReaderFactory 구현:

```
pub struct SyncStateReaderFactory {
    shared_state_sync_client: SharedStateSyncClient,
    class_manager_client: SharedClassManagerClient,
    runtime: tokio::runtime::Handle,
}

impl StateReaderFactory for SyncStateReaderFactory {
    fn get_state_reader(&self) -> Box<dyn StateReader> {
        // 1. 최신 상태 동기화 확인
        // 2. 상태 읽기 인터페이스 생성
        // 3. 클래스 매니저와 연동
    }
}
```

3.4.7 에러 처리 및 응답 생성

에러 매핑:

```
fn convert_compiled_class_hash_error(error: ValidateCompiledClassHashError) -> S
    StarknetError {
        code: StarknetErrorCode::UnknownErrorCode(
            "StarknetErrorCode.INVALID_COMPILED_CLASS_HASH".to_string(),
        ),
        message: error.to_string(),
    }
}
```

Gateway 출력 생성:

```
fn create_gateway_output(internal_rpc_tx: &InternalRpcTransaction) -> GatewayOut
    let transaction_hash = internal_rpc_tx.tx_hash;
    match &internal_rpc_tx.tx {
        InternalRpcTransactionWithoutTxHash::Declare(declare_tx) => {
            GatewayOutput::Declare(DeclareGatewayOutput::new(transaction_hash, d
        }
        InternalRpcTransactionWithoutTxHash::DeployAccount(deploy_account_tx) =>
            GatewayOutput::DeployAccount(DeployAccountGatewayOutput::new(
                transaction_hash,
                deploy_account_tx.contract_address,
            ))
        }
        InternalRpcTransactionWithoutTxHash::Invoke(_) => {
            GatewayOutput::Invoke(InvokeGatewayOutput::new(transaction_hash))
        }
    }
}
```

3.4.8 성능 최적화

비동기 처리:

- CPU 집약적 작업을 `spawn_blocking` 으로 분리
- 논블로킹 I/O 작업 최적화
- 동시 요청 처리 지원

메트릭스 수집:

- 트랜잭션 처리 지연시간 측정
- 성공/실패율 추적
- P2P vs 직접 제출 구분 통계

캐싱 전략:

- 상태 읽기 결과 캐싱
- 클래스 정보 캐싱
- 검증 결과 임시 저장

3.5 Batcher (apollo_batcher)

역할: 트랜잭션을 배치로 묶어 블록을 생성하고 실행하는 핵심 컴포넌트

3.5.1 Batcher 핵심 구조

```rust
pub struct Batcher {
    pub config: BatcherConfig,
    // 저장소 읽기/쓰기 인터페이스
    pub storage_reader: Arc<dyn BatcherStorageReaderTrait>,
    pub storage_writer: Box<dyn BatcherStorageWriterTrait>,
    // 외부 서비스 클라이언트들
    pub l1_provider_client: SharedL1ProviderClient,
    pub mempool_client: SharedMempoolClient,
    pub transaction_converter: TransactionConverter,

    // 블록 빌더 팩토리 (테스트 가능한 설계)
    block_builder_factory: Box<dyn BlockBuilderFactoryTrait>,
    pre_confirmed_block_writer_factory: Box<dyn PreconfirmedBlockWriterFactoryTr

    // 현재 작업 중인 높이
    active_height: Option<BlockNumber>,
    // 활성 제안 관리
    active_proposal: Arc<Mutex<Option<ProposalId>>>,
    active_proposal_task: Option<ProposalTask>,
    // 실행 완료된 제안들
    executed_proposals: Arc<Mutex<HashMap<ProposalId, ProposalResult<BlockExecut

    // 트랜잭션 스트림 관리
    propose_tx_streams: HashMap<ProposalId, OutputStreamReceiver>,
    validate_tx_streams: HashMap<ProposalId, InputStreamSender>,
}
```

3.5.2 높이 관리 및 제안 생성

높이 시작 과정:

```rust
pub async fn start_height(&mut self, input: StartHeightInput) -> BatcherResult<(
    // 1. 중복 높이 검사
    if self.active_height == Some(input.height) {
        return Err(BatcherError::HeightInProgress);
    }

    // 2. 저장소 높이 마커 검증
    let storage_height = self.get_height_from_storage()?;
    if storage_height != input.height {
        return Err(BatcherError::StorageHeightMarkerMismatch {
```

```
            marker_height: storage_height,
            requested_height: input.height,
        });
    }

    // 3. 이전 높이 작업 중단
    self.abort_active_height().await;

    // 4. 새 높이 설정
    self.active_height = Some(input.height);
    Ok(())
}
```

3.5.3 블록 제안 과정

제안 생성 (Propose Block):

```
pub async fn propose_block(&mut self, propose_block_input: ProposeBlockInput) ->
    let block_number = propose_block_input.block_info.block_number;
    let active_height = self.active_height.ok_or(BatcherError::NoActiveHeight)?;

    // 1. 블록 입력 검증
    verify_block_input(active_height, block_number, propose_block_input.retrospe

    // 2. 외부 서비스 준비
    // Mempool 커밋 준비
    self.mempool_client.commit_block(CommitBlockArgs::default()).await?;
    // 가스 가격 업데이트
    self.mempool_client.update_gas_price(
        propose_block_input.block_info.gas_prices.strk_gas_prices.l2_gas_price.g
    ).await?;
    // L1 Provider 세션 시작
    self.l1_provider_client.start_block(
        SessionState::Propose,
        propose_block_input.block_info.block_number
    ).await?;

    // 3. 트랜잭션 제공자 생성
    let tx_provider = ProposeTransactionProvider::new(
        self.mempool_client.clone(),
        self.l1_provider_client.clone(),
        self.config.max_l1_handler_txs_per_block_proposal,
        propose_block_input.block_info.block_number,
    );
```

```
    // 4. 출력 스트림 설정
    let (output_tx_sender, output_tx_receiver) = tokio::sync::mpsc::unbounded_ch

    // 5. Pre-confirmed 블록 라이터 생성
    let cende_block_metadata = CendeBlockMetadata::new(propose_block_input.block
    let (pre_confirmed_block_writer, candidate_tx_sender, pre_confirmed_tx_sende
        self.pre_confirmed_block_writer_factory.create(
            propose_block_input.block_info.block_number,
            propose_block_input.proposal_round,
            cende_block_metadata,
        );

    // 6. 블록 빌더 생성 및 실행
    let (block_builder, abort_signal_sender) = self.block_builder_factory.create
        BlockMetadata {
            block_info: propose_block_input.block_info,
            retrospective_block_hash: propose_block_input.retrospective_block_ha
        },
        BlockBuilderExecutionParams {
            deadline: deadline_as_instant(propose_block_input.deadline)?,
            is_validator: false,
        },
        Box::new(tx_provider),
        Some(output_tx_sender),
        Some(candidate_tx_sender),
        Some(pre_confirmed_tx_sender),
        tokio::runtime::Handle::current(),
    )?;

    // 7. 제안 실행 시작
    self.spawn_proposal(
        propose_block_input.proposal_id,
        block_builder,
        abort_signal_sender,
        None,
        Some(pre_confirmed_block_writer),
        ProposalMetricsHandle::new(),
    ).await?;

    // 8. 출력 스트림 등록
    self.propose_tx_streams.insert(propose_block_input.proposal_id, output_tx_re
    Ok(())
}
```

3.5.4 블록 검증 과정

검증 실행 (Validate Block):

```
pub async fn validate_block(&mut self, validate_block_input: ValidateBlockInput)
    // 1. L1 Provider 검증 세션 시작
    self.l1_provider_client.start_block(
        SessionState::Validate,
        validate_block_input.block_info.block_number
    ).await?;

    // 2. 입력 스트림 설정
    let (input_tx_sender, input_tx_receiver) =
        tokio::sync::mpsc::channel(self.config.input_stream_content_buffer_size)
    let (final_n_executed_txs_sender, final_n_executed_txs_receiver) =
        tokio::sync::oneshot::channel();

    // 3. 검증용 트랜잭션 제공자 생성
    let tx_provider = ValidateTransactionProvider::new(
        input_tx_receiver,
        final_n_executed_txs_receiver,
        self.l1_provider_client.clone(),
        validate_block_input.block_info.block_number,
    );

    // 4. 검증용 블록 빌더 생성
    let (block_builder, abort_signal_sender) = self.block_builder_factory.create
        BlockMetadata {
            block_info: validate_block_input.block_info,
            retrospective_block_hash: validate_block_input.retrospective_block_h
        },
        BlockBuilderExecutionParams {
            deadline: deadline_as_instant(validate_block_input.deadline)?,
            is_validator: true,
        },
        Box::new(tx_provider),
        None, None, None,
        tokio::runtime::Handle::current(),
    )?;

    // 5. 검증 실행 시작
    self.spawn_proposal(
        validate_block_input.proposal_id,
        block_builder,
        abort_signal_sender,
        Some(final_n_executed_txs_sender),
        None,
```

```
        ProposalMetricsHandle::new(),
    ).await?;

    // 6. 입력 스트림 등록
    self.validate_tx_streams.insert(validate_block_input.proposal_id, input_tx_s
    Ok(())
}
```

3.5.5 제안 콘텐츠 관리

트랜잭션 전송 처리:

```
async fn handle_send_txs_request(
    &mut self,
    proposal_id: ProposalId,
    txs: Vec<InternalConsensusTransaction>,
) -> BatcherResult<SendProposalContentResponse> {
    if self.is_active(proposal_id).await {
        // 활성 제안인 경우: 트랜잭션을 제공자에게 전송
        let tx_provider_sender = &self.validate_tx_streams.get(&proposal_id)
            .expect("Expecting tx_provider_sender to exist during batching.");

        for tx in txs {
            tx_provider_sender.send(tx).await.map_err(|err| {
                error!("Failed to send transaction to the tx provider: {}", err)
                BatcherError::InternalError
            })?;
        }
        return Ok(SendProposalContentResponse { response: ProposalStatus::Proces
    }

    // 비활성 제안인 경우: 완료된 결과 반환
    let proposal_result = self.get_completed_proposal_result(proposal_id).await
        .expect("Proposal should exist.");
    match proposal_result {
        Ok(_) => panic!("Proposal finished validation before all transactions we
        Err(err) => Ok(SendProposalContentResponse { response: proposal_status_f
    }
}
```

3.5.6 블록 커밋 및 상태 업데이트

제안 및 블록 커밋:

```
async fn commit_proposal_and_block(
    &mut self,
    height: BlockNumber,
    state_diff: ThinStateDiff,
    address_to_nonce: HashMap<ContractAddress, Nonce>,
    consumed_l1_handler_tx_hashes: IndexSet<TransactionHash>,
    rejected_tx_hashes: IndexSet<TransactionHash>,
) -> BatcherResult<()> {
    // 1. 저장소에 제안 커밋
    self.storage_writer.commit_proposal(height, state_diff.clone()).map_err(|err
        error!("Failed to commit proposal to storage: {}", err);
        BatcherError::InternalError
    })?;

    // 2. L1 Provider에 블록 커밋
    let rejected_l1_handler_tx_hashes = rejected_tx_hashes.iter().copied()
        .filter(|tx_hash| consumed_l1_handler_tx_hashes.contains(tx_hash))
        .collect();

    let l1_provider_result = self.l1_provider_client.commit_block(
        consumed_l1_handler_tx_hashes,
        rejected_l1_handler_tx_hashes,
        height,
    ).await;

    // L1 Provider 커밋 실패 시 롤백
    if let Err(err) = l1_provider_result {
        error!("L1 provider commit failed: {:?}", err);
        self.storage_writer.revert_block(height);
        return Err(BatcherError::InternalError);
    }

    // 3. Mempool에 블록 커밋
    let mempool_result = self.mempool_client.commit_block(CommitBlockArgs {
        address_to_nonce,
        rejected_tx_hashes,
    }).await;

    if let Err(mempool_err) = mempool_result {
        error!("Failed to commit block to mempool: {}", mempool_err);
        // Mempool 커밋 실패는 복구 가능한 오류로 처리
    }
```

```
    Ok(())
}
```

3.5.7 동기화 블록 처리

Sync Block 추가:

```
pub async fn add_sync_block(&mut self, sync_block: SyncBlock) -> BatcherResult<(
    let SyncBlock {
        state_diff,
        account_transaction_hashes,
        l1_transaction_hashes,
        block_header_without_hash: BlockHeaderWithoutHash { block_number, .. },
    } = sync_block;

    // 1. 높이 검증
    let height = self.get_height_from_storage()?;
    if height != block_number {
        return Err(BatcherError::StorageHeightMarkerMismatch {
            marker_height: height,
            requested_height: block_number,
        });
    }

    // 2. 활성 작업 중단
    if let Some(height) = self.active_height {
        info!("Aborting all work on height {} due to state sync.", height);
        self.abort_active_height().await;
    }

    // 3. 동기화된 블록 커밋
    let address_to_nonce = state_diff.nonces.iter().map(|(k, v)| (*k, *v)).colle
    self.commit_proposal_and_block(
        height,
        state_diff,
        address_to_nonce,
        l1_transaction_hashes.iter().copied().collect(),
        Default::default(),
    ).await?;

    // 4. 메트릭 업데이트
    LAST_SYNCED_BLOCK.set_lossy(block_number.0);
    SYNCED_TRANSACTIONS.increment(
        (account_transaction_hashes.len() + l1_transaction_hashes.len()).try_int
    );
```

```
    Ok(())
}
```

3.5.8 결정 도달 처리

합의 결정 처리:

```
pub async fn decision_reached(&mut self, input: DecisionReachedInput) -> Batcher
    let height = self.active_height.ok_or(BatcherError::NoActiveHeight)?;
    let proposal_id = input.proposal_id;

    // 1. 실행된 제안 결과 가져오기
    let proposal_result = self.executed_proposals.lock().await.remove(&proposal_
        .ok_or(BatcherError::ExecutedProposalNotFound { proposal_id })?
        .map_err(|err| {
            error!("Failed to get block execution artifacts: {}", err);
            BatcherError::InternalError
        })?;

    // 2. 실행 결과에서 상태 차이 추출
    let state_diff = proposal_result.thin_state_diff();
    let execution_infos = proposal_result.execution_data.execution_infos;

    // 3. 제안 및 블록 커밋
    self.commit_proposal_and_block(
        height,
        state_diff.clone(),
        proposal_result.address_to_nonce(),
        proposal_result.execution_data.consumed_l1_handler_tx_hashes,
        proposal_result.execution_data.rejected_tx_hashes,
    ).await?;

    // 4. 메트릭 업데이트
    let n_txs = u64::try_from(proposal_result.tx_hashes().len()).unwrap();
    LAST_BATCHED_BLOCK.set_lossy(height.0);
    BATCHED_TRANSACTIONS.increment(n_txs);

    // 5. 응답 생성
    Ok(DecisionReachedResponse {
        state_diff,
        l2_gas_used: proposal_result.l2_gas_used,
        central_objects: CentralObjects {
            execution_infos,
```

```
            bouncer_weights: proposal_result.bouncer_weights,
            compressed_state_diff: proposal_result.compressed_state_diff,
            // ... 기타 실행 결과 데이터
        },
    })
}
```

3.5.9 성능 최적화 및 모니터링

메트릭스 수집:

- `LAST_PROPOSED_BLOCK` : 마지막 제안된 블록 높이
- `LAST_BATCHED_BLOCK` : 마지막 배치된 블록 높이
- `LAST_SYNCED_BLOCK` : 마지막 동기화된 블록 높이
- `BATCHED_TRANSACTIONS` : 배치된 트랜잭션 수
- `REJECTED_TRANSACTIONS` : 거부된 트랜잭션 수
- `REVERTED_TRANSACTIONS` : 되돌려진 트랜잭션 수

동시성 관리:

- `Arc<Mutex<>>` 를 사용한 안전한 상태 공유
- 비동기 태스크 스폰을 통한 병렬 처리
- 중단 신호를 통한 그레이스풀 종료

4. 실행 엔진 (Blockifier)

4.1 Blockifier 개요

Blockifier는 Starknet 트랜잭션의 실제 실행을 담당하는 핵심 엔진으로, Cairo 프로그램의 실행과 상태 관리를 담당합니다.

4.1.1 핵심 모듈 구조

```rust
// Blockifier 주요 모듈들
pub mod abi;                    // ABI 인코딩/디코딩
pub mod blockifier;             // 메인 블록 실행기
pub mod bouncer;                // 리소스 제한 관리
pub mod concurrency;            // 병렬 실행 지원
pub mod context;                // 실행 컨텍스트
```

```rust
pub mod execution;              // 트랜잭션 실행 로직
pub mod fee;                    // 수수료 계산
pub mod state;                  // 상태 관리
pub mod transaction;            // 트랜잭션 처리
```

4.1.2 주요 기능

트랜잭션 실행:

- Cairo 프로그램 실행 및 검증
- 스마트 컨트랙트 호출 처리
- 시스템 호출 (syscall) 구현
- 실행 결과 및 이벤트 생성

상태 관리:

- Patricia Tree 기반 상태 저장
- 상태 변경 사항 추적 및 적용
- 상태 커밋 및 롤백 지원
- 상태 스냅샷 관리

리소스 관리:

- 가스 계산 및 수수료 처리
- 실행 리소스 제한 (Bouncer)
- 메모리 및 CPU 사용량 모니터링

4.2 트랜잭션 실행 과정

4.2.1 실행 파이프라인

```rust
pub struct TransactionExecutor {
    // 실행 컨텍스트
    context: BlockContext,
    // 상태 관리자
    state: Box<dyn State>,
    // 리소스 추적기
    bouncer: Bouncer,
}

impl TransactionExecutor {
    pub fn execute_transaction(
```

```
        &mut self,
        tx: &ExecutableTransaction,
    ) -> TransactionExecutionResult<TransactionExecutionInfo> {
        // 1. 전처리 단계
        self.pre_process_transaction(tx)?;

        // 2. 실행 단계
        let execution_result = self.execute_transaction_inner(tx)?;

        // 3. 후처리 단계
        self.post_process_transaction(tx, &execution_result)?;

        // 4. 결과 반환
        Ok(execution_result)
    }
}
```

4.2.2 전처리 단계 (Pre-processing)

1. 트랜잭션 유효성 검증:

```
fn pre_process_transaction(&mut self, tx: &ExecutableTransaction) -> Result<(),
    // 서명 검증
    self.validate_signature(tx)?;

    // Nonce 검증
    self.validate_nonce(tx)?;

    // 가스 한도 검증
    self.validate_gas_limits(tx)?;

    // 계정 잔액 검증
    self.validate_account_balance(tx)?;

    Ok(())
}
```

2. 실행 컨텍스트 설정:

- 블록 정보 (높이, 타임스탬프, 시퀀서 주소)
- 체인 ID 및 네트워크 설정
- 가스 가격 및 수수료 설정
- 버전별 상수 (Versioned Constants)

4.2.3 실행 단계 (Execution)

Cairo VM 실행:

```
fn execute_transaction_inner(&mut self, tx: &ExecutableTransaction) -> Execution
    match tx {
        ExecutableTransaction::Invoke(invoke_tx) => {
            // 1. 계정 컨트랙트 호출
            let call_info = self.execute_account_transaction(invoke_tx)?;

            // 2. 실제 함수 호출 실행
            let validate_call_info = self.execute_validate_call(invoke_tx)?;
            let execute_call_info = self.execute_execute_call(invoke_tx)?;

            // 3. 수수료 전송 처리
            let fee_transfer_info = self.execute_fee_transfer(invoke_tx)?;

            Ok(TransactionExecutionInfo {
                validate_call_info,
                execute_call_info,
                fee_transfer_info,
                actual_fee: self.calculate_actual_fee()?,
                actual_resources: self.get_actual_resources(),
            })
        }
        ExecutableTransaction::Declare(declare_tx) => {
            // Declare 트랜잭션 실행 로직
            self.execute_declare_transaction(declare_tx)
        }
        ExecutableTransaction::DeployAccount(deploy_tx) => {
            // DeployAccount 트랜잭션 실행 로직
            self.execute_deploy_account_transaction(deploy_tx)
        }
    }
}
```

시스템 호출 처리:

```
// Cairo 프로그램에서 호출되는 시스템 호출들
pub enum SyscallName {
    CallContract,       // 다른 컨트랙트 호출
    Deploy,            // 새 컨트랙트 배포
    EmitEvent,         // 이벤트 발생
    GetBlockHash,      // 블록 해시 조회
```

```
    GetExecutionInfo, // 실행 정보 조회
    LibraryCall,      // 라이브러리 호출
    StorageRead,      // 저장소 읽기
    StorageWrite,     // 저장소 쓰기
    // ... 기타 시스템 호출들
}
```

4.2.4 후처리 단계 (Post-processing)

상태 변경 사항 적용:

```
fn post_process_transaction(
    &mut self,
    tx: &ExecutableTransaction,
    execution_info: &TransactionExecutionInfo,
) -> Result<(), TransactionExecutionError> {
    // 1. 상태 변경 사항 커밋
    self.state.commit_changes(&execution_info.state_changes)?;

    // 2. 이벤트 로그 추가
    self.add_events_to_log(&execution_info.events)?;

    // 3. 수수료 차감 및 전송
    self.process_fee_payment(tx, &execution_info.actual_fee)?;

    // 4. Nonce 증가
    self.increment_account_nonce(tx.sender_address())?;

    Ok(())
}
```

4.3 상태 관리 시스템

4.3.1 Patricia Tree 구현

상태 트리 구조:

```
pub struct PatriciaTree {
    // 루트 노드
    root: Option<NodeId>,
    // 노드 저장소
    nodes: HashMap<NodeId, PatriciaNode>,
    // 수정된 노드들 (커밋 대기)
```

```
    modified_nodes: HashSet<NodeId>,
}

pub enum PatriciaNode {
    // 내부 노드 (분기점)
    Internal {
        left: Option<NodeId>,
        right: Option<NodeId>,
        hash: Felt,
    },
    // 리프 노드 (실제 값)
    Leaf {
        key: Felt,
        value: Felt,
        hash: Felt,
    },
}
```

상태 읽기/쓰기 인터페이스:

```
pub trait StateReader {
    fn get_storage_at(&self, contract_address: ContractAddress, key: StorageKey)
    fn get_nonce_at(&self, contract_address: ContractAddress) -> StateResult<Non
    fn get_class_hash_at(&self, contract_address: ContractAddress) -> StateResul
    fn get_compiled_contract_class(&self, class_hash: ClassHash) -> StateResult<
}

pub trait StateWriter: StateReader {
    fn set_storage_at(&mut self, contract_address: ContractAddress, key: Storage
    fn increment_nonce(&mut self, contract_address: ContractAddress) -> StateRes
    fn set_class_hash_at(&mut self, contract_address: ContractAddress, class_has
    fn set_compiled_contract_class(&mut self, class_hash: ClassHash, compiled_cl
}
```

4.3.2 상태 캐싱 전략

다층 캐시 구조:

```
pub struct CachedState<S: StateReader> {
    // 기본 상태 읽기 인터페이스
    state_reader: S,
    // 저장소 캐시
    storage_cache: HashMap<(ContractAddress, StorageKey), Felt>,
```

```
    // Nonce 캐시
    nonce_cache: HashMap<ContractAddress, Nonce>,
    // 클래스 해시 캐시
    class_hash_cache: HashMap<ContractAddress, ClassHash>,
    // 컴파일된 클래스 캐시
    compiled_class_cache: HashMap<ClassHash, CompiledContractClass>,
}
```

4.4 병렬 실행 지원

4.4.1 동시성 모델

워커 풀 기반 병렬 처리:

```
pub struct WorkerPool {
    // 워커 스레드들
    workers: Vec<Worker>,
    // 작업 큐
    task_queue: Arc<Mutex<VecDeque<ExecutionTask>>>,
    // 결과 수집기
    result_collector: Arc<Mutex<HashMap<TransactionHash, ExecutionResult>>>,
}

pub struct ExecutionTask {
    transaction: ExecutableTransaction,
    read_set: HashSet<StorageKey>,
    write_set: HashSet<StorageKey>,
    dependencies: Vec<TransactionHash>,
}
```

충돌 감지 및 해결:

```
fn detect_conflicts(tasks: &[ExecutionTask]) -> Vec<ConflictGroup> {
    let mut conflicts = Vec::new();

    for (i, task1) in tasks.iter().enumerate() {
        for (j, task2) in tasks.iter().enumerate().skip(i + 1) {
            // 읽기-쓰기 충돌 검사
            if task1.write_set.intersection(&task2.read_set).next().is_some() ||
               task2.write_set.intersection(&task1.read_set).next().is_some() ||
               task1.write_set.intersection(&task2.write_set).next().is_some() {
                conflicts.push(ConflictGroup { tasks: vec![i, j] });
            }
```

```
        }
    }

    conflicts
}
```

4.5 수수료 계산 시스템

4.5.1 리소스 추적

실행 리소스 타입:

```
pub struct ExecutionResources {
    // Cairo 단계 수
    n_steps: usize,
    // 메모리 홀 수
    n_memory_holes: usize,
    // 내장 함수 사용량
    builtin_instance_counter: HashMap<String, usize>,
    // 시스템 호출 수
    syscall_counter: HashMap<SyscallName, usize>,
}
```

가스 계산 로직:

```
pub fn calculate_tx_gas_usage(
    resources: &ExecutionResources,
    gas_costs: &GasCosts,
) -> Result<GasAmount, FeeError> {
    let mut total_gas = GasAmount(0);

    // Cairo 단계 비용
    total_gas += GasAmount(resources.n_steps * gas_costs.step_gas_cost);

    // 메모리 홀 비용
    total_gas += GasAmount(resources.n_memory_holes * gas_costs.memory_hole_gas_

    // 내장 함수 비용
    for (builtin_name, count) in &resources.builtin_instance_counter {
        let builtin_cost = gas_costs.builtin_gas_costs.get(builtin_name)
            .ok_or(FeeError::UnknownBuiltin(builtin_name.clone()))?;
        total_gas += GasAmount(count * builtin_cost);
    }
```

```
    Ok(total_gas)
}
```

4.6 에러 처리 및 복구

4.6.1 실행 에러 타입

```
#[derive(Debug, Error)]
pub enum TransactionExecutionError {
    #[error("Transaction validation failed: {0}")]
    ValidationFailure(String),

    #[error("Insufficient account balance")]
    InsufficientAccountBalance,

    #[error("Contract not found: {address}")]
    ContractNotFound { address: ContractAddress },

    #[error("Execution reverted: {revert_reason}")]
    ExecutionReverted { revert_reason: String },

    #[error("Out of gas")]
    OutOfGas,

    #[error("State error: {0}")]
    StateError(StateError),
}
```

4.6.2 복구 메커니즘

상태 롤백:

```
impl<S: StateReader> CachedState<S> {
    pub fn create_checkpoint(&self) -> StateCheckpoint {
        StateCheckpoint {
            storage_cache: self.storage_cache.clone(),
            nonce_cache: self.nonce_cache.clone(),
            class_hash_cache: self.class_hash_cache.clone(),
        }
    }

    pub fn rollback_to_checkpoint(&mut self, checkpoint: StateCheckpoint) {
```

```
        self.storage_cache = checkpoint.storage_cache;
        self.nonce_cache = checkpoint.nonce_cache;
        self.class_hash_cache = checkpoint.class_hash_cache;
    }
}
```

5. L1 연동 시스템

5.1 L1 Provider (apollo_l1_provider)

역할: Ethereum L1과의 상호작용을 관리하여 L2의 최종성을 보장하는 핵심 컴포넌트

5.1.1 L1 Provider 구조

```
pub struct L1Provider {
    config: L1ProviderConfig,
    // L1 블록체인과의 통신 클라이언트
    l1_provider_client: SharedL1ProviderClient,
    // 배치 처리 클라이언트
    batcher_client: SharedBatcherClient,
    // 상태 동기화 클라이언트
    state_sync_client: SharedStateSyncClient,
    // 현재 처리 중인 L2 높이
    current_l2_height: BlockNumber,
    // L1에서 증명된 마지막 L2 블록
    last_proved_l2_block: BlockNumber,
}

pub struct L1ProviderBuilder {
    config: L1ProviderConfig,
    l1_provider_client: SharedL1ProviderClient,
    batcher_client: SharedBatcherClient,
    state_sync_client: SharedStateSyncClient,
    startup_height: Option<BlockNumber>,
    catchup_height: Option<BlockNumber>,
}
```

5.1.2 L1 블록 모니터링

L1 Scraper 연동:

```rust
pub struct L1Scraper<T: BaseLayerContract> {
    config: L1ScraperConfig,
    l1_provider_client: SharedL1ProviderClient,
    base_layer_contract: T,
    // 추적할 이벤트 식별자들
    event_identifiers: Vec<EventIdentifier>,
    // 마지막 처리된 L1 블록
    last_l1_block_processed: BlockNumber,
}

impl<T: BaseLayerContract> L1Scraper<T> {
    pub async fn scrape_l1_events(&mut self) -> Result<Vec<L1Event>, L1ScraperEr
        // 1. 새로운 L1 블록들 확인
        let latest_l1_block = self.base_layer_contract.get_latest_block().await?

        // 2. 이벤트 스크래핑 범위 결정
        let from_block = self.last_l1_block_processed + 1;
        let to_block = latest_l1_block.number;

        // 3. 관심 있는 이벤트들 필터링
        let events = self.base_layer_contract
            .get_events(from_block, to_block, &self.event_identifiers)
            .await?;

        // 4. 이벤트 파싱 및 분류
        let parsed_events = self.parse_and_classify_events(events)?;

        // 5. 처리된 블록 높이 업데이트
        self.last_l1_block_processed = to_block;

        Ok(parsed_events)
    }
}
```

5.1.3 상태 루트 제출 과정

L2 상태 루트 제출:

```rust
pub async fn submit_state_root(
    &self,
    l2_block_number: BlockNumber,
    state_root: Felt,
    proof: StateUpdateProof,
) -> Result<TransactionHash, L1ProviderError> {
```

```
    // 1. 제출 조건 검증
    self.validate_submission_conditions(l2_block_number, &state_root)?;

    // 2. L1 트랜잭션 데이터 구성
    let submission_data = StateUpdateSubmission {
        program_output: ProgramOutput {
            prev_state_root: self.get_previous_state_root(l2_block_number - 1)?,
            new_state_root: state_root,
            block_number: l2_block_number,
            block_hash: self.get_l2_block_hash(l2_block_number)?,
        },
        proof,
    };

    // 3. L1 컨트랙트 호출
    let tx_hash = self.base_layer_contract
        .update_state(submission_data)
        .await?;

    // 4. 제출 상태 추적
    self.track_submission_status(tx_hash, l2_block_number).await?;

    Ok(tx_hash)
}
```

5.1.4 L1↔L2 메시지 처리

L1 → L2 메시지 처리:

```
pub struct L1HandlerTransaction {
    // L1에서 발생한 이벤트 정보
    from_address: EthereumAddress,
    to_address: ContractAddress,
    entry_point_selector: EntryPointSelector,
    calldata: Vec<Felt>,
    // L1 트랜잭션 정보
    l1_tx_hash: H256,
    l1_block_number: u64,
    l1_tx_index: u64,
}

impl L1Provider {
    pub async fn process_l1_to_l2_messages(
        &mut self,
        l1_events: Vec<L1Event>,
```

```
    ) -> Result<Vec<L1HandlerTransaction>, L1ProviderError> {
        let mut l1_handler_txs = Vec::new();

        for event in l1_events {
            match event.event_type {
                L1EventType::LogMessageToL2 => {
                    // 1. 이벤트 데이터 파싱
                    let message_data = self.parse_l1_to_l2_message(&event)?;

                    // 2. L1 Handler 트랜잭션 생성
                    let l1_handler_tx = L1HandlerTransaction {
                        from_address: message_data.from_address,
                        to_address: message_data.to_address,
                        entry_point_selector: message_data.selector,
                        calldata: message_data.payload,
                        l1_tx_hash: event.transaction_hash,
                        l1_block_number: event.block_number,
                        l1_tx_index: event.transaction_index,
                    };

                    l1_handler_txs.push(l1_handler_tx);
                }
                L1EventType::LogMessageToL1 => {
                    // L2 → L1 메시지 처리 (소비 확인)
                    self.process_l2_to_l1_message_consumption(&event)?;
                }
                _ => continue,
            }
        }

        Ok(l1_handler_txs)
    }
}
```

L2 → L1 메시지 처리:

```
pub async fn consume_l2_to_l1_message(
    &self,
    message: L2ToL1Message,
    merkle_proof: Vec<Felt>,
) -> Result<TransactionHash, L1ProviderError> {
    // 1. 메시지 해시 계산
    let message_hash = self.calculate_message_hash(&message)?;

    // 2. 머클 증명 검증
```

```
    self.verify_message_inclusion_proof(&message_hash, &merkle_proof)?;

    // 3. L1 컨트랙트에서 메시지 소비
    let tx_hash = self.base_layer_contract
        .consume_message_from_l2(message, merkle_proof)
        .await?;

    Ok(tx_hash)
}
```

5.1.5 세션 상태 관리

제안/검증 세션:

```
#[derive(Debug, Clone, PartialEq)]
pub enum SessionState {
    Propose,   // 블록 제안 세션
    Validate,  // 블록 검증 세션
}

impl L1Provider {
    pub async fn start_block(
        &mut self,
        session_state: SessionState,
        block_number: BlockNumber,
    ) -> Result<(), L1ProviderError> {
        // 1. 세션 상태 검증
        self.validate_session_transition(session_state, block_number)?;

        // 2. L1 Handler 트랜잭션 준비
        let l1_handler_txs = match session_state {
            SessionState::Propose => {
                // 제안 모드: 새로운 L1 Handler 트랜잭션들 수집
                self.collect_pending_l1_handler_txs(block_number).await?
            }
            SessionState::Validate => {
                // 검증 모드: 기존 L1 Handler 트랜잭션들 검증
                self.validate_l1_handler_txs(block_number).await?
            }
        };

        // 3. 세션 상태 업데이트
        self.current_session = Some(BlockSession {
            state: session_state,
            block_number,
```

```
            l1_handler_txs,
            start_time: Instant::now(),
        });

        Ok(())
    }

    pub async fn commit_block(
        &mut self,
        consumed_l1_handler_tx_hashes: IndexSet<TransactionHash>,
        rejected_l1_handler_tx_hashes: Vec<TransactionHash>,
        block_number: BlockNumber,
    ) -> Result<(), L1ProviderError> {
        // 1. 세션 검증
        let session = self.current_session.take()
            .ok_or(L1ProviderError::NoActiveSession)?;

        if session.block_number != block_number {
            return Err(L1ProviderError::UnexpectedHeight {
                expected_height: session.block_number,
                got: block_number,
            });
        }

        // 2. L1 Handler 트랜잭션 상태 업데이트
        self.update_l1_handler_tx_status(
            &consumed_l1_handler_tx_hashes,
            &rejected_l1_handler_tx_hashes,
        ).await?;

        // 3. 다음 블록 높이로 진행
        self.current_l2_height = block_number + 1;

        Ok(())
    }
}
```

5.2 L1 Gas Price (apollo_l1_gas_price)

역할: L1 가스 가격을 모니터링하고 L2 수수료 계산에 반영

5.2.1 가스 가격 제공자 (L1GasPriceProvider)

```rust
pub struct L1GasPriceProvider {
    config: L1GasPriceProviderConfig,
    // 현재 가스 가격 정보
    current_gas_prices: Arc<RwLock<L1GasPrices>>,
    // 가격 히스토리 (예측용)
    price_history: VecDeque<TimestampedGasPrice>,
}

#[derive(Debug, Clone)]
pub struct L1GasPrices {
    // ETH 가스 가격 (wei/gas)
    eth_gas_price: GasPrice,
    // STRK 가스 가격 (fri/gas)
    strk_gas_price: GasPrice,
    // 마지막 업데이트 시간
    last_update: DateTime<Utc>,
    // 데이터 소스
    source: GasPriceSource,
}

impl L1GasPriceProvider {
    pub async fn get_current_gas_prices(&self) -> L1GasPrices {
        self.current_gas_prices.read().await.clone()
    }

    pub async fn update_gas_prices(&mut self, new_prices: L1GasPrices) -> Result
        // 1. 가격 유효성 검증
        self.validate_gas_prices(&new_prices)?;

        // 2. 급격한 변동 검사
        self.check_price_volatility(&new_prices)?;

        // 3. 가격 업데이트
        {
            let mut current_prices = self.current_gas_prices.write().await;
            *current_prices = new_prices.clone();
        }

        // 4. 히스토리 업데이트
        self.price_history.push_back(TimestampedGasPrice {
            prices: new_prices,
            timestamp: Utc::now(),
        });

        // 5. 오래된 히스토리 정리
```

```
        self.cleanup_old_history();

        Ok(())
    }
}
```

5.2.2 가스 가격 스크래퍼 (L1GasPriceScraper)

```
pub struct L1GasPriceScraper<T: BaseLayerContract> {
    config: L1GasPriceScraperConfig,
    l1_gas_price_client: SharedL1GasPriceClient,
    base_layer_contract: T,
    // 가격 수집 간격
    scraping_interval: Duration,
}

impl<T: BaseLayerContract> L1GasPriceScraper<T> {
    pub async fn scrape_gas_prices(&mut self) -> Result<(), L1GasPriceScraperErr
        // 1. L1에서 현재 가스 가격 조회
        let eth_gas_price = self.base_layer_contract.get_gas_price().await?;

        // 2. 여러 소스에서 가격 정보 수집
        let oracle_prices = self.fetch_oracle_gas_prices().await?;
        let dex_prices = self.fetch_dex_gas_prices().await?;

        // 3. 가격 정보 집계 및 검증
        let aggregated_prices = self.aggregate_gas_prices(
            eth_gas_price,
            oracle_prices,
            dex_prices,
        )?;

        // 4. L1 가스 가격 제공자에 업데이트
        self.l1_gas_price_client
            .update_gas_prices(aggregated_prices)
            .await?;

        Ok(())
    }

    async fn aggregate_gas_prices(
        &self,
        eth_price: GasPrice,
        oracle_prices: Vec<GasPrice>,
```

```
        dex_prices: Vec<GasPrice>,
    ) -> Result<L1GasPrices, L1GasPriceScraperError> {
        // 1. 가중 평균 계산
        let weighted_eth_price = self.calculate_weighted_average(&[
            (eth_price, self.config.l1_source_weight),
            (oracle_prices.iter().sum::<GasPrice>() / oracle_prices.len(), self.
            (dex_prices.iter().sum::<GasPrice>() / dex_prices.len(), self.config
        ])?;

        // 2. STRK 가격 계산 (ETH/STRK 환율 적용)
        let eth_strk_rate = self.get_eth_strk_exchange_rate().await?;
        let strk_gas_price = GasPrice::from(weighted_eth_price.0 * eth_strk_rate

        Ok(L1GasPrices {
            eth_gas_price: weighted_eth_price,
            strk_gas_price,
            last_update: Utc::now(),
            source: GasPriceSource::Aggregated,
        })
    }
}
```

5.3 Base Layer Integration

5.3.1 Ethereum Base Layer Contract

```
pub struct EthereumBaseLayerContract {
    config: BaseLayerConfig,
    // Ethereum 클라이언트
    eth_client: Arc<Provider<Http>>,
    // Starknet Core 컨트랙트 주소
    core_contract_address: Address,
    // 컨트랙트 ABI
    contract_abi: Abi,
}

impl BaseLayerContract for EthereumBaseLayerContract {
    async fn get_latest_block(&self) -> Result<L1Block, BaseLayerError> {
        let block = self.eth_client
            .get_block(BlockNumber::Latest)
            .await?
            .ok_or(BaseLayerError::BlockNotFound)?;
```

```
    Ok(L1Block {
        number: block.number.unwrap().as_u64(),
        hash: block.hash.unwrap(),
        timestamp: block.timestamp.as_u64(),
        parent_hash: block.parent_hash,
    })
}

async fn get_proved_block_at(&self, l1_height: u64) -> Result<L2Block, BaseL
    // 1. L1 블록에서 상태 업데이트 이벤트 조회
    let state_update_events = self.get_state_update_events_at_height(l1_heig

    // 2. 가장 최근 상태 업데이트 선택
    let latest_update = state_update_events
        .into_iter()
        .max_by_key(|event| event.l2_block_number)
        .ok_or(BaseLayerError::NoStateUpdateFound)?;

    Ok(L2Block {
        number: latest_update.l2_block_number,
        state_root: latest_update.new_state_root,
        block_hash: latest_update.block_hash,
    })
}

async fn update_state(&self, submission: StateUpdateSubmission) -> Result<H2
    // 1. 트랜잭션 데이터 인코딩
    let tx_data = self.encode_state_update_call(&submission)?;

    // 2. 가스 추정
    let gas_estimate = self.eth_client
        .estimate_gas(&TransactionRequest {
            to: Some(self.core_contract_address),
            data: Some(tx_data.clone()),
            ..Default::default()
        }, None)
        .await?;

    // 3. 트랜잭션 전송
    let tx_hash = self.eth_client
        .send_transaction(TransactionRequest {
            to: Some(self.core_contract_address),
            data: Some(tx_data),
            gas: Some(gas_estimate * 120 / 100), // 20% 버퍼
            gas_price: Some(self.get_gas_price().await?),
            ..Default::default()
```

```
            }, None)
            .await?;

        Ok(tx_hash)
    }
}
```

5.3.2 모니터링된 Base Layer

```
pub struct MonitoredEthereumBaseLayer {
    base_layer: EthereumBaseLayerContract,
    l1_endpoint_monitor: SharedL1EndpointMonitorClient,
    endpoint_url: String,
}

impl BaseLayerContract for MonitoredEthereumBaseLayer {
    async fn get_latest_block(&self) -> Result<L1Block, BaseLayerError> {
        // 1. 엔드포인트 상태 확인
        self.l1_endpoint_monitor
            .check_endpoint_health(&self.endpoint_url)
            .await?;

        // 2. 실제 요청 실행
        let result = self.base_layer.get_latest_block().await;

        // 3. 결과에 따른 모니터링 업데이트
        match &result {
            Ok(_) => {
                self.l1_endpoint_monitor
                    .record_successful_request(&self.endpoint_url)
                    .await?;
            }
            Err(err) => {
                self.l1_endpoint_monitor
                    .record_failed_request(&self.endpoint_url, err.to_string())
                    .await?;
            }
        }

        result
    }
}
```

5.3.3 이벤트 추적 시스템

```rust
pub fn event_identifiers_to_track() -> Vec<EventIdentifier> {
    vec![
        // 상태 업데이트 이벤트
        EventIdentifier {
            contract_address: STARKNET_CORE_CONTRACT,
            event_signature: "LogStateUpdate(uint256,bytes32,bytes32)",
        },
        // L1 → L2 메시지 이벤트
        EventIdentifier {
            contract_address: STARKNET_CORE_CONTRACT,
            event_signature: "LogMessageToL2(address,uint256,uint256,uint256,uin
        },
        // L2 → L1 메시지 소비 이벤트
        EventIdentifier {
            contract_address: STARKNET_CORE_CONTRACT,
            event_signature: "ConsumedMessageToL1(address,address,uint256[])",
        },
    ]
}

pub async fn fetch_start_block(
    base_layer: &EthereumBaseLayerContract,
    config: &L1ScraperConfig,
) -> Result<u64, L1ScraperError> {
    match config.start_block_override {
        Some(override_block) => {
            info!("Using L1 scraper start block override: {}", override_block);
            Ok(override_block)
        }
        None => {
            // 1. 최신 블록 조회
            let latest_block = base_layer.get_latest_block().await?;

            // 2. 설정된 블록 수만큼 뒤로 이동
            let start_block = latest_block.number
                .saturating_sub(config.blocks_behind_latest);

            info!("Calculated L1 scraper start block: {}", start_block);
            Ok(start_block)
        }
    }
}
```

6. 네트워킹 및 P2P

6.1 P2P 네트워크 (apollo_network)

역할: 분산 네트워크에서 노드 간 통신 및 데이터 전파를 담당하는 핵심 인프라

6.1.1 네트워크 아키텍처

libp2p 기반 구현:

```rust
pub struct P2PNetwork {
    // libp2p Swarm 인스턴스
    swarm: Swarm<NetworkBehaviour>,
    // 로컬 피어 ID
    local_peer_id: PeerId,
    // 연결된 피어들
    connected_peers: HashMap<PeerId, PeerInfo>,
    // 네트워크 설정
    config: NetworkConfig,
    // 메시지 핸들러들
    message_handlers: HashMap<ProtocolId, Box<dyn MessageHandler>>,
}

#[derive(NetworkBehaviour)]
pub struct NetworkBehaviour {
    // 피어 발견 프로토콜
    kademlia: Kademlia<MemoryStore>,
    // 가십 프로토콜 (트랜잭션/블록 전파)
    gossipsub: Gossipsub,
    // 직접 요청/응답 프로토콜
    request_response: RequestResponse<GenericCodec>,
    // 연결 관리
    connection_limits: ConnectionLimits,
    // 피어 식별
    identify: Identify,
}
```

6.1.2 피어 발견 및 연결 관리

부트스트랩 과정:

```rust
impl P2PNetwork {
    pub async fn bootstrap(&mut self) -> Result<(), NetworkError> {
        // 1. 부트스트랩 노드들에 연결
        for bootstrap_addr in &self.config.bootstrap_nodes {
            self.dial_peer(bootstrap_addr).await?;
        }

        // 2. DHT 부트스트랩 시작
        self.swarm.behaviour_mut().kademlia.bootstrap()?;

        // 3. 피어 발견 시작
        self.start_peer_discovery().await?;

        // 4. 주기적 연결 유지 작업 시작
        self.start_connection_maintenance().await;

        Ok(())
    }

    async fn start_peer_discovery(&mut self) -> Result<(), NetworkError> {
        // Kademlia를 통한 피어 발견
        let query_id = self.swarm
            .behaviour_mut()
            .kademlia
            .get_closest_peers(self.local_peer_id);

        // 발견된 피어들과 연결 시도
        self.handle_peer_discovery_results(query_id).await?;

        Ok(())
    }
}
```

연결 품질 관리:

```rust
#[derive(Debug, Clone)]
pub struct PeerInfo {
    peer_id: PeerId,
    // 연결 품질 점수
    reputation_score: i32,
    // 마지막 활동 시간
    last_seen: Instant,
    // 지원하는 프로토콜들
    supported_protocols: Vec<ProtocolId>,
    // 연결 통계
```

```rust
    connection_stats: ConnectionStats,
}

impl P2PNetwork {
    fn update_peer_reputation(&mut self, peer_id: &PeerId, delta: i32) {
        if let Some(peer_info) = self.connected_peers.get_mut(peer_id) {
            peer_info.reputation_score += delta;

            // 평판이 너무 낮으면 연결 해제
            if peer_info.reputation_score < self.config.min_reputation_threshold
                self.disconnect_peer(peer_id);
            }
        }
    }

    async fn maintain_connections(&mut self) {
        // 1. 비활성 피어들 정리
        self.cleanup_inactive_peers().await;

        // 2. 연결 수 조절
        self.balance_connection_count().await;

        // 3. 새로운 피어 발견 시도
        if self.connected_peers.len() < self.config.target_peer_count {
            self.discover_new_peers().await;
        }
    }
}
```

6.1.3 메시지 전파 시스템

GossipSub 프로토콜:

```rust
pub enum NetworkMessage {
    // 새로운 트랜잭션
    NewTransaction(InternalRpcTransaction),
    // 새로운 블록 헤더
    NewBlockHeader(BlockHeader),
    // 합의 메시지
    ConsensusMessage(ConsensusMessage),
    // 상태 동기화 요청
    StateSyncRequest(StateSyncRequest),
}

impl P2PNetwork {
```

```rust
pub async fn broadcast_message(
    &mut self,
    topic: &str,
    message: NetworkMessage,
) -> Result<(), NetworkError> {
    // 1. 메시지 직렬화
    let serialized = self.serialize_message(&message)?;

    // 2. 중복 방지를 위한 해시 계산
    let message_hash = self.calculate_message_hash(&serialized);

    // 3. 이미 전파된 메시지인지 확인
    if self.is_message_already_seen(&message_hash) {
        return Ok(());
    }

    // 4. GossipSub을 통해 전파
    self.swarm
        .behaviour_mut()
        .gossipsub
        .publish(IdentTopic::new(topic), serialized)?;

    // 5. 전파된 메시지 기록
    self.mark_message_as_seen(message_hash);

    Ok(())
}

pub async fn handle_incoming_message(
    &mut self,
    peer_id: PeerId,
    topic: String,
    data: Vec<u8>,
) -> Result<(), NetworkError> {
    // 1. 메시지 역직렬화
    let message = self.deserialize_message(&data)?;

    // 2. 메시지 유효성 검증
    self.validate_message(&message, &peer_id)?;

    // 3. 적절한 핸들러로 전달
    match topic.as_str() {
        "transactions" => {
            self.handle_transaction_message(message, peer_id).await?;
        }
        "blocks" => {
```

```
                self.handle_block_message(message, peer_id).await?;
            }
            "consensus" => {
                self.handle_consensus_message(message, peer_id).await?;
            }
            _ => {
                warn!("Unknown topic: {}", topic);
            }
        }

        // 4. 피어 평판 업데이트
        self.update_peer_reputation(&peer_id, 1);

        Ok(())
    }
}
```

6.1.4 Mempool P2P 전파

트랜잭션 전파 메커니즘:

```
pub struct MempoolP2pPropagator {
    network_client: Arc<dyn NetworkClient>,
    config: MempoolP2pConfig,
    // 전파된 트랜잭션 추적
    propagated_transactions: LruCache<TransactionHash, Instant>,
    // 전파 통계
    propagation_stats: PropagationStats,
}

impl MempoolP2pPropagator {
    pub async fn propagate_transaction(
        &mut self,
        tx: &InternalRpcTransaction,
        metadata: Option<BroadcastedMessageMetadata>,
    ) -> Result<(), PropagationError> {
        // 1. 이미 전파된 트랜잭션인지 확인
        if self.propagated_transactions.contains(&tx.tx_hash) {
            return Ok(());
        }

        // 2. 전파 조건 검사
        if !self.should_propagate_transaction(tx, &metadata)? {
            return Ok(());
        }
```

```
        // 3. 트랜잭션 직렬화
        let serialized_tx = self.serialize_transaction(tx)?;

        // 4. P2P 네트워크로 전파
        self.network_client
            .broadcast_message("transactions", serialized_tx)
            .await?;

        // 5. 전파 기록 업데이트
        self.propagated_transactions.put(tx.tx_hash, Instant::now());
        self.propagation_stats.increment_propagated_count();

        Ok(())
    }

    fn should_propagate_transaction(
        &self,
        tx: &InternalRpcTransaction,
        metadata: &Option<BroadcastedMessageMetadata>,
    ) -> Result<bool, PropagationError> {
        // 1. P2P에서 받은 트랜잭션은 재전파하지 않음
        if metadata.is_some() {
            return Ok(false);
        }

        // 2. 수수료가 최소 임계값 이상인지 확인
        if tx.tip().0 < self.config.min_propagation_fee {
            return Ok(false);
        }

        // 3. 트랜잭션 크기 제한 확인
        if tx.total_bytes() > self.config.max_transaction_size {
            return Ok(false);
        }

        Ok(true)
    }
}

pub struct MempoolP2pRunner {
    network_client: Arc<dyn NetworkClient>,
    gateway_client: SharedGatewayClient,
    class_manager_client: SharedClassManagerClient,
    propagator_client: SharedMempoolP2pPropagatorClient,
}
```

```rust
impl MempoolP2pRunner {
    pub async fn run(&mut self) -> Result<(), P2pRunnerError> {
        let mut message_stream = self.network_client.subscribe_to_topic("transac

        while let Some(message) = message_stream.next().await {
            match self.handle_received_transaction(message).await {
                Ok(_) => {
                    // 성공적으로 처리됨
                }
                Err(err) => {
                    warn!("Failed to handle received transaction: {}", err);
                    // 에러 메트릭 업데이트
                }
            }
        }

        Ok(())
    }

    async fn handle_received_transaction(
        &mut self,
        message: NetworkMessage,
    ) -> Result<(), P2pRunnerError> {
        // 1. 메시지에서 트랜잭션 추출
        let (tx, peer_id) = self.extract_transaction_from_message(message)?;

        // 2. 기본 유효성 검증
        self.validate_received_transaction(&tx)?;

        // 3. Gateway를 통해 트랜잭션 제출
        let metadata = BroadcastedMessageMetadata {
            originator_peer_id: peer_id,
            received_timestamp: Utc::now(),
        };

        match self.gateway_client.add_tx(tx, Some(metadata)).await {
            Ok(_) => {
                // 성공적으로 추가됨
                self.update_peer_reputation(&peer_id, 5);
            }
            Err(err) => {
                // 추가 실패 (중복, 유효하지 않음 등)
                warn!("Failed to add received transaction: {}", err);
                self.update_peer_reputation(&peer_id, -1);
            }
```

```
        }

        Ok(())
    }
}
```

6.2 State Sync (apollo_state_sync)

역할: 네트워크의 최신 상태와 동기화를 유지하여 노드가 항상 최신 블록체인 상태를 반영하도록 함

6.2.1 State Sync 구조

```rust
pub struct StateSync {
    config: StateSyncConfig,
    // 클래스 매니저 클라이언트
    class_manager_client: SharedClassManagerClient,
    // 현재 동기화 상태
    sync_status: Arc<RwLock<SyncStatus>>,
    // 동기화 통계
    sync_metrics: SyncMetrics,
}

pub struct StateSyncRunner {
    state_sync: Arc<StateSync>,
    // P2P 네트워크 클라이언트
    network_client: Arc<dyn NetworkClient>,
    // 동기화 작업 큐
    sync_tasks: VecDeque<SyncTask>,
    // 동기화 진행 상태
    sync_progress: SyncProgress,
}

#[derive(Debug, Clone)]
pub enum SyncStatus {
    // 동기화되지 않음
    NotSynced { behind_by: u64 },
    // 동기화 진행 중
    Syncing {
        current_height: BlockNumber,
        target_height: BlockNumber,
        sync_rate: f64, // blocks/second
    },
    // 동기화 완료
```

```
    Synced { last_sync_time: DateTime<Utc> },
    // 동기화 실패
    Failed { error: String, retry_count: u32 },
}
```

6.2.2 빠른 상태 동기화 (Fast Sync)

스냅샷 기반 동기화:

```
impl StateSyncRunner {
    pub async fn fast_sync_to_height(&mut self, target_height: BlockNumber) -> R
        // 1. 신뢰할 수 있는 스냅샷 소스 찾기
        let snapshot_sources = self.discover_snapshot_sources(target_height).awa

        // 2. 스냅샷 다운로드 및 검증
        let state_snapshot = self.download_and_verify_snapshot(
            target_height,
            &snapshot_sources,
        ).await?;

        // 3. 로컬 상태를 스냅샷으로 교체
        self.apply_state_snapshot(state_snapshot).await?;

        // 4. 스냅샷 이후 블록들 동기화
        self.sync_blocks_from_height(target_height + 1).await?;

        Ok(())
    }

    async fn download_and_verify_snapshot(
        &mut self,
        height: BlockNumber,
        sources: &[PeerId],
    ) -> Result<StateSnapshot, StateSyncError> {
        for source_peer in sources {
            match self.download_snapshot_from_peer(height, source_peer).await {
                Ok(snapshot) => {
                    // 스냅샷 무결성 검증
                    if self.verify_snapshot_integrity(&snapshot).await? {
                        return Ok(snapshot);
                    } else {
                        warn!("Invalid snapshot from peer: {}", source_peer);
                        self.penalize_peer(source_peer).await;
                    }
                }
```

```
                Err(err) => {
                    warn!("Failed to download snapshot from peer {}: {}", source
                    continue;
                }
            }
        }

        Err(StateSyncError::NoValidSnapshotFound)
    }
}
```

6.2.3 블록 동기화 과정

순차적 블록 다운로드:

```
impl StateSyncRunner {
    pub async fn sync_blocks_from_height(&mut self, start_height: BlockNumber) -
        let mut current_height = start_height;
        let target_height = self.get_network_tip_height().await?;

        while current_height <= target_height {
            // 1. 배치 크기 결정
            let batch_size = self.calculate_optimal_batch_size(current_height, t
            let end_height = (current_height + batch_size - 1).min(target_height

            // 2. 블록 배치 다운로드
            let blocks = self.download_block_batch(current_height, end_height).a

            // 3. 블록 검증 및 적용
            for block in blocks {
                self.validate_and_apply_block(block).await?;
            }

            // 4. 진행 상황 업데이트
            current_height = end_height + 1;
            self.update_sync_progress(current_height, target_height).await;

            // 5. 동기화 속도 조절
            self.throttle_sync_if_needed().await;
        }

        Ok(())
    }
```

```rust
    async fn download_block_batch(
        &mut self,
        start_height: BlockNumber,
        end_height: BlockNumber,
    ) -> Result<Vec<SyncBlock>, StateSyncError> {
        let mut blocks = Vec::new();
        let available_peers = self.get_available_sync_peers().await;

        // 피어들에게 블록 요청 분산
        let requests_per_peer = (end_height - start_height + 1) / available_peer

        let mut tasks = Vec::new();
        for (i, peer) in available_peers.iter().enumerate() {
            let peer_start = start_height + (i as u64 * requests_per_peer);
            let peer_end = if i == available_peers.len() - 1 {
                end_height
            } else {
                peer_start + requests_per_peer - 1
            };

            let task = self.request_blocks_from_peer(*peer, peer_start, peer_end
            tasks.push(task);
        }

        // 모든 요청 완료 대기
        let results = futures::future::join_all(tasks).await;

        // 결과 수집 및 정렬
        for result in results {
            match result {
                Ok(peer_blocks) => blocks.extend(peer_blocks),
                Err(err) => {
                    warn!("Failed to download blocks from peer: {}", err);
                    // 실패한 블록들은 다른 피어에서 재시도
                }
            }
        }

        // 높이 순으로 정렬
        blocks.sort_by_key(|block| block.block_header_without_hash.block_number)

        Ok(blocks)
    }
}
```

6.2.4 상태 검증 시스템

블록 검증 과정:

```
impl StateSyncRunner {
    async fn validate_and_apply_block(&mut self, sync_block: SyncBlock) -> Resul
        // 1. 블록 헤더 검증
        self.validate_block_header(&sync_block.block_header_without_hash).await?

        // 2. 상태 차이 검증
        self.validate_state_diff(&sync_block.state_diff).await?;

        // 3. 트랜잭션 해시 검증
        self.validate_transaction_hashes(
            &sync_block.account_transaction_hashes,
            &sync_block.l1_transaction_hashes,
        ).await?;

        // 4. 클래스 선언 검증
        self.validate_declared_classes(&sync_block.state_diff).await?;

        // 5. 상태 변경 사항 적용
        self.apply_state_diff(&sync_block.state_diff).await?;

        // 6. 블록 커밋
        self.commit_block(sync_block).await?;

        Ok(())
    }

    async fn validate_state_diff(&self, state_diff: &ThinStateDiff) -> Result<()
        // 1. 저장소 업데이트 검증
        for (address, storage_updates) in &state_diff.storage_diffs {
            for (key, value) in storage_updates {
                // 저장소 키/값 유효성 검사
                self.validate_storage_entry(*address, *key, *value)?;
            }
        }

        // 2. Nonce 업데이트 검증
        for (address, nonce) in &state_diff.nonces {
            self.validate_nonce_update(*address, *nonce)?;
        }

        // 3. 클래스 해시 업데이트 검증
```

```
        for (address, class_hash) in &state_diff.deployed_contracts {
            self.validate_class_hash_update(*address, *class_hash).await?;
        }

        // 4. 선언된 클래스 검증
        for declared_class in &state_diff.declared_classes {
            self.validate_declared_class(declared_class).await?;
        }

        Ok(())
    }

    async fn validate_declared_classes(&self, state_diff: &ThinStateDiff) -> Res
        for declared_class in &state_diff.declared_classes {
            // 1. 클래스 매니저에서 클래스 정보 조회
            let class_info = self.class_manager_client
                .get_class_info(declared_class.class_hash)
                .await?;

            // 2. 클래스 해시 일치 확인
            if class_info.class_hash != declared_class.class_hash {
                return Err(StateSyncError::ClassHashMismatch {
                    expected: declared_class.class_hash,
                    actual: class_info.class_hash,
                });
            }

            // 3. 컴파일된 클래스 해시 확인 (Cairo 1.0+)
            if let Some(compiled_class_hash) = declared_class.compiled_class_has
                if class_info.compiled_class_hash != Some(compiled_class_hash) {
                    return Err(StateSyncError::CompiledClassHashMismatch {
                        expected: compiled_class_hash,
                        actual: class_info.compiled_class_hash,
                    });
                }
            }
        }

        Ok(())
    }
}
```

6.2.5 동기화 상태 관리

동기화 진행 상황 추적:

```rust
#[derive(Debug, Clone)]
pub struct SyncProgress {
    // 시작 높이
    start_height: BlockNumber,
    // 현재 높이
    current_height: BlockNumber,
    // 목표 높이
    target_height: BlockNumber,
    // 동기화 시작 시간
    start_time: Instant,
    // 마지막 업데이트 시간
    last_update: Instant,
    // 동기화 속도 (블록/초)
    sync_rate: f64,
}

impl SyncProgress {
    pub fn calculate_eta(&self) -> Duration {
        let remaining_blocks = self.target_height - self.current_height;
        if self.sync_rate > 0.0 {
            Duration::from_secs_f64(remaining_blocks as f64 / self.sync_rate)
        } else {
            Duration::from_secs(u64::MAX) // 무한대
        }
    }

    pub fn completion_percentage(&self) -> f64 {
        let total_blocks = self.target_height - self.start_height;
        let completed_blocks = self.current_height - self.start_height;

        if total_blocks > 0 {
            (completed_blocks as f64 / total_blocks as f64) * 100.0
        } else {
            100.0
        }
    }
}

impl StateSyncRunner {
    async fn update_sync_progress(&mut self, current_height: BlockNumber, target
        let now = Instant::now();
        let elapsed = now.duration_since(self.sync_progress.last_update);
```

```
        if elapsed.as_secs() > 0 {
            let blocks_synced = current_height - self.sync_progress.current_heig
            let new_rate = blocks_synced as f64 / elapsed.as_secs_f64();

            // 지수 이동 평균으로 동기화 속도 평활화
            self.sync_progress.sync_rate =
                0.7 * self.sync_progress.sync_rate + 0.3 * new_rate;
        }

        self.sync_progress.current_height = current_height;
        self.sync_progress.target_height = target_height;
        self.sync_progress.last_update = now;

        // 동기화 상태 업데이트
        let sync_status = if current_height >= target_height {
            SyncStatus::Synced { last_sync_time: Utc::now() }
        } else {
            SyncStatus::Syncing {
                current_height,
                target_height,
                sync_rate: self.sync_progress.sync_rate,
            }
        };

        *self.state_sync.sync_status.write().await = sync_status;
    }
}
```

7. 컴파일 및 클래스 관리

7.1 Class Manager (apollo_class_manager)

역할: Starknet 컨트랙트 클래스의 전체 라이프사이클을 관리하는 중앙 집중식 서비스

7.1.1 Class Manager 구조

```
pub struct ClassManager {
    config: ClassManagerConfig,
    // Sierra 컴파일러 클라이언트
    sierra_compiler_client: SharedSierraCompilerClient,
    // 클래스 저장소
```

```
    class_storage: Arc<dyn ClassStorage>,
    // 클래스 캐시
    class_cache: Arc<RwLock<LruCache<ClassHash, CachedClassInfo>>>,
    // 컴파일 작업 큐
    compilation_queue: Arc<Mutex<VecDeque<CompilationTask>>>,
    // 컴파일 통계
    compilation_metrics: CompilationMetrics,
}

#[derive(Debug, Clone)]
pub struct CachedClassInfo {
    // 클래스 해시
    class_hash: ClassHash,
    // 컴파일된 클래스 해시 (Cairo 1.0+)
    compiled_class_hash: Option<CompiledClassHash>,
    // Sierra 프로그램 (Cairo 1.0+)
    sierra_program: Option<SierraProgram>,
    // CASM 프로그램
    casm_program: CasmProgram,
    // 클래스 ABI
    abi: ContractClassAbi,
    // 캐시 생성 시간
    cached_at: Instant,
}
```

7.1.2 클래스 선언 처리

Declare 트랜잭션 처리 과정:

```
impl ClassManager {
    pub async fn handle_declare_transaction(
        &mut self,
        declare_tx: &DeclareTransaction,
    ) -> Result<ClassInfo, ClassManagerError> {
        match declare_tx {
            DeclareTransaction::V0(declare_v0) => {
                self.handle_cairo0_declare(declare_v0).await
            }
            DeclareTransaction::V1(declare_v1) => {
                self.handle_cairo0_declare_v1(declare_v1).await
            }
            DeclareTransaction::V2(declare_v2) => {
                self.handle_cairo1_declare_v2(declare_v2).await
            }
            DeclareTransaction::V3(declare_v3) => {
```

```
            self.handle_cairo1_declare_v3(declare_v3).await
        }
    }
}

async fn handle_cairo1_declare_v2(
    &mut self,
    declare_v2: &DeclareTransactionV2,
) -> Result<ClassInfo, ClassManagerError> {
    // 1. Sierra 클래스 검증
    let sierra_class = &declare_v2.contract_class;
    self.validate_sierra_class(sierra_class)?;

    // 2. 클래스 해시 계산 및 검증
    let calculated_class_hash = self.calculate_class_hash(sierra_class)?;
    if calculated_class_hash != declare_v2.class_hash {
        return Err(ClassManagerError::ClassHashMismatch {
            expected: declare_v2.class_hash,
            calculated: calculated_class_hash,
        });
    }

    // 3. Sierra → CASM 컴파일
    let compilation_result = self.sierra_compiler_client
        .compile_sierra_to_casm(sierra_class.clone())
        .await?;

    // 4. 컴파일된 클래스 해시 검증
    let calculated_compiled_hash = self.calculate_compiled_class_hash(&compi
    if calculated_compiled_hash != declare_v2.compiled_class_hash {
        return Err(ClassManagerError::CompiledClassHashMismatch {
            expected: declare_v2.compiled_class_hash,
            calculated: calculated_compiled_hash,
        });
    }

    // 5. 클래스 정보 저장
    let class_info = ClassInfo {
        class_hash: declare_v2.class_hash,
        compiled_class_hash: Some(declare_v2.compiled_class_hash),
        sierra_program: Some(sierra_class.sierra_program.clone()),
        casm_program: compilation_result.casm_class,
        abi: sierra_class.abi.clone(),
    };

    self.store_class_info(&class_info).await?;
```

```
        // 6. 캐시 업데이트
        self.update_class_cache(&class_info).await;

        Ok(class_info)
    }
}
```

7.1.3 클래스 해시 계산

클래스 해시 계산 알고리즘:

```
impl ClassManager {
    fn calculate_class_hash(&self, sierra_class: &SierraClass) -> Result<ClassHa
        // 1. Sierra 프로그램 정규화
        let normalized_program = self.normalize_sierra_program(&sierra_class.sie

        // 2. ABI 정규화
        let normalized_abi = self.normalize_abi(&sierra_class.abi)?;

        // 3. 엔트리 포인트 정규화
        let normalized_entry_points = self.normalize_entry_points(&sierra_class.

        // 4. 해시 계산을 위한 데이터 구성
        let hash_data = ClassHashData {
            sierra_program: normalized_program,
            abi: normalized_abi,
            entry_points: normalized_entry_points,
        };

        // 5. Pedersen 해시 계산
        let class_hash = self.compute_pedersen_hash(&hash_data)?;

        Ok(ClassHash(class_hash))
    }

    fn calculate_compiled_class_hash(&self, casm_class: &CasmClass) -> Result<Co
        // 1. CASM 바이트코드 정규화
        let normalized_bytecode = self.normalize_casm_bytecode(&casm_class.bytec

        // 2. 엔트리 포인트 정규화
        let normalized_entry_points = self.normalize_casm_entry_points(&casm_cla

        // 3. 해시 계산
```

```
        let compiled_hash_data = CompiledClassHashData {
            bytecode: normalized_bytecode,
            entry_points: normalized_entry_points,
        };

        let compiled_class_hash = self.compute_poseidon_hash(&compiled_hash_data

        Ok(CompiledClassHash(compiled_class_hash))
    }
}
```

7.1.4 클래스 저장 및 조회

클래스 저장소 인터페이스:

```
pub trait ClassStorage: Send + Sync {
    async fn store_class(&self, class_hash: ClassHash, class_info: &ClassInfo) -
    async fn get_class(&self, class_hash: ClassHash) -> Result<Option<ClassInfo>
    async fn get_compiled_class(&self, compiled_class_hash: CompiledClassHash) -
    async fn class_exists(&self, class_hash: ClassHash) -> Result<bool, StorageE
    async fn list_classes(&self, limit: usize, offset: usize) -> Result<Vec<Clas
}

pub struct DatabaseClassStorage {
    db_pool: Arc<DatabasePool>,
    compression_enabled: bool,
}

impl ClassStorage for DatabaseClassStorage {
    async fn store_class(&self, class_hash: ClassHash, class_info: &ClassInfo) -
        let mut conn = self.db_pool.get_connection().await?;

        // 1. 클래스 정보 직렬화
        let serialized_sierra = class_info.sierra_program.as_ref()
            .map(|p| self.serialize_and_compress(p))
            .transpose()?;
        let serialized_casm = self.serialize_and_compress(&class_info.casm_progr
        let serialized_abi = self.serialize_and_compress(&class_info.abi)?;

        // 2. 데이터베이스에 저장
        conn.execute(
            "INSERT INTO classes (class_hash, compiled_class_hash, sierra_progra
             VALUES ($1, $2, $3, $4, $5, $6)
             ON CONFLICT (class_hash) DO NOTHING",
```

```
            &[
                &class_hash.0.to_bytes_be(),
                &class_info.compiled_class_hash.map(|h| h.0.to_bytes_be()),
                &serialized_sierra,
                &serialized_casm,
                &serialized_abi,
                &Utc::now(),
            ],
        ).await?;

        Ok(())
    }

    async fn get_class(&self, class_hash: ClassHash) -> Result<Option<ClassInfo>
        let mut conn = self.db_pool.get_connection().await?;

        let row = conn.query_opt(
            "SELECT compiled_class_hash, sierra_program, casm_program, abi FROM
            &[&class_hash.0.to_bytes_be()],
        ).await?;

        if let Some(row) = row {
            // 데이터 역직렬화 및 압축 해제
            let compiled_class_hash: Option<Vec<u8>> = row.get(0);
            let sierra_program: Option<Vec<u8>> = row.get(1);
            let casm_program: Vec<u8> = row.get(2);
            let abi: Vec<u8> = row.get(3);

            let class_info = ClassInfo {
                class_hash,
                compiled_class_hash: compiled_class_hash
                    .map(|bytes| CompiledClassHash(Felt::from_bytes_be(&bytes)))
                sierra_program: sierra_program
                    .map(|bytes| self.decompress_and_deserialize(&bytes))
                    .transpose()?,
                casm_program: self.decompress_and_deserialize(&casm_program)?,
                abi: self.decompress_and_deserialize(&abi)?,
            };

            Ok(Some(class_info))
        } else {
            Ok(None)
        }
    }
}
```

7.1.5 클래스 캐시 관리

LRU 캐시 구현:

```
impl ClassManager {
    async fn get_class_info(&self, class_hash: ClassHash) -> Result<ClassInfo, C
        // 1. 캐시에서 먼저 확인
        {
            let cache = self.class_cache.read().await;
            if let Some(cached_info) = cache.get(&class_hash) {
                // 캐시 히트 메트릭 업데이트
                self.compilation_metrics.cache_hits.increment(1);
                return Ok(cached_info.clone().into());
            }
        }

        // 2. 캐시 미스 - 저장소에서 조회
        self.compilation_metrics.cache_misses.increment(1);
        let class_info = self.class_storage
            .get_class(class_hash)
            .await?
            .ok_or(ClassManagerError::ClassNotFound { class_hash })?;

        // 3. 캐시에 추가
        self.update_class_cache(&class_info).await;

        Ok(class_info)
    }

    async fn update_class_cache(&self, class_info: &ClassInfo) {
        let mut cache = self.class_cache.write().await;

        let cached_info = CachedClassInfo {
            class_hash: class_info.class_hash,
            compiled_class_hash: class_info.compiled_class_hash,
            sierra_program: class_info.sierra_program.clone(),
            casm_program: class_info.casm_program.clone(),
            abi: class_info.abi.clone(),
            cached_at: Instant::now(),
        };

        cache.put(class_info.class_hash, cached_info);
    }

    // 주기적 캐시 정리
```

```rust
    pub async fn cleanup_expired_cache_entries(&self) {
        let mut cache = self.class_cache.write().await;
        let now = Instant::now();
        let expiry_duration = Duration::from_secs(self.config.cache_ttl_seconds)

        cache.retain(|_, cached_info| {
            now.duration_since(cached_info.cached_at) < expiry_duration
        });
    }
}
```

7.2 Sierra Compiler (apollo_compile_to_casm)

역할: Sierra 중간 표현을 CASM (Cairo Assembly) 바이트코드로 컴파일하는 핵심 컴파일러

7.2.1 Sierra Compiler 구조

```rust
pub struct SierraCompiler {
    config: SierraCompilerConfig,
    // 컴파일 작업 풀
    compilation_pool: Arc<ThreadPool>,
    // 컴파일 캐시
    compilation_cache: Arc<RwLock<LruCache<SierraHash, CompilationResult>>>,
    // 컴파일 통계
    compilation_stats: CompilationStats,
}

#[derive(Debug, Clone)]
pub struct CompilationResult {
    // 컴파일된 CASM 클래스
    casm_class: CasmClass,
    // 컴파일 메타데이터
    compilation_metadata: CompilationMetadata,
    // 컴파일 시간
    compilation_time: Duration,
    // 최적화 레벨
    optimization_level: OptimizationLevel,
}

#[derive(Debug, Clone)]
pub struct CompilationMetadata {
    // Sierra 프로그램 크기
    sierra_program_size: usize,
```

```
    // 생성된 CASM 명령어 수
    casm_instruction_count: usize,
    // 사용된 내장 함수들
    used_builtins: Vec<String>,
    // 컴파일러 버전
    compiler_version: String,
}
```

7.2.2 Sierra → CASM 컴파일 과정

컴파일 파이프라인:

```
impl SierraCompiler {
    pub async fn compile_sierra_to_casm(
        &self,
        sierra_program: SierraProgram,
    ) -> Result<CompilationResult, CompilationError> {
        let start_time = Instant::now();

        // 1. Sierra 프로그램 해시 계산 (캐시 키)
        let sierra_hash = self.calculate_sierra_hash(&sierra_program)?;

        // 2. 캐시 확인
        if let Some(cached_result) = self.get_cached_compilation(&sierra_hash).a
            self.compilation_stats.cache_hits.increment(1);
            return Ok(cached_result);
        }

        // 3. 실제 컴파일 수행
        self.compilation_stats.cache_misses.increment(1);
        let compilation_result = self.perform_compilation(sierra_program).await?

        // 4. 컴파일 시간 기록
        let compilation_time = start_time.elapsed();
        let mut result = compilation_result;
        result.compilation_time = compilation_time;

        // 5. 캐시에 저장
        self.cache_compilation_result(sierra_hash, &result).await;

        // 6. 통계 업데이트
        self.compilation_stats.total_compilations.increment(1);
        self.compilation_stats.total_compilation_time.increment(compilation_time

        Ok(result)
```

```
}

async fn perform_compilation(
    &self,
    sierra_program: SierraProgram,
) -> Result<CompilationResult, CompilationError> {
    // CPU 집약적 작업을 별도 스레드에서 실행
    let config = self.config.clone();
    let result = tokio::task::spawn_blocking(move || {
        Self::compile_sierra_program_blocking(sierra_program, config)
    }).await.map_err(|e| CompilationError::ThreadPanic(e.to_string()))??;

    Ok(result)
}

fn compile_sierra_program_blocking(
    sierra_program: SierraProgram,
    config: SierraCompilerConfig,
) -> Result<CompilationResult, CompilationError> {
    // 1. Sierra 프로그램 파싱 및 검증
    let parsed_program = Self::parse_sierra_program(&sierra_program)?;
    Self::validate_sierra_program(&parsed_program)?;

    // 2. 타입 체크 및 분석
    let type_info = Self::analyze_types(&parsed_program)?;
    let control_flow = Self::analyze_control_flow(&parsed_program)?;

    // 3. CASM 코드 생성
    let mut casm_generator = CasmGenerator::new(config.optimization_level);
    let casm_instructions = casm_generator.generate_casm(&parsed_program, &t

    // 4. 최적화 적용
    let optimized_instructions = Self::optimize_casm(casm_instructions, &con

    // 5. 엔트리 포인트 생성
    let entry_points = Self::generate_entry_points(&parsed_program, &optimiz

    // 6. CASM 클래스 구성
    let casm_class = CasmClass {
        prime: CAIRO_PRIME.clone(),
        compiler_version: env!("CARGO_PKG_VERSION").to_string(),
        bytecode: optimized_instructions,
        entry_points_by_type: entry_points,
        hints: Self::extract_hints(&parsed_program)?,
    };
```

```
        // 7. 컴파일 메타데이터 생성
        let metadata = CompilationMetadata {
            sierra_program_size: sierra_program.len(),
            casm_instruction_count: casm_class.bytecode.len(),
            used_builtins: Self::extract_used_builtins(&casm_class),
            compiler_version: env!("CARGO_PKG_VERSION").to_string(),
        };

        Ok(CompilationResult {
            casm_class,
            compilation_metadata: metadata,
            compilation_time: Duration::default(), // 나중에 설정됨
            optimization_level: config.optimization_level,
        })
    }
}
```

7.2.3 CASM 코드 생성

명령어 생성 과정:

```
struct CasmGenerator {
    optimization_level: OptimizationLevel,
    instruction_buffer: Vec<CasmInstruction>,
    label_map: HashMap<String, usize>,
    builtin_usage: HashMap<String, usize>,
}

impl CasmGenerator {
    fn generate_casm(
        &mut self,
        sierra_program: &ParsedSierraProgram,
        type_info: &TypeInfo,
    ) -> Result<Vec<CasmInstruction>, CompilationError> {
        // 1. 함수별 코드 생성
        for function in &sierra_program.functions {
            self.generate_function_code(function, type_info)?;
        }

        // 2. 라이브러리 함수 코드 생성
        for lib_func in &sierra_program.libfuncs {
            self.generate_libfunc_code(lib_func, type_info)?;
        }
```

```rust
    // 3. 초기화 코드 생성
    self.generate_initialization_code(sierra_program)?;

    // 4. 정리 코드 생성
    self.generate_cleanup_code(sierra_program)?;

    Ok(self.instruction_buffer.clone())
}

fn generate_function_code(
    &mut self,
    function: &SierraFunction,
    type_info: &TypeInfo,
) -> Result<(), CompilationError> {
    // 함수 시작 라벨
    let function_label = format!("func_{}", function.id);
    self.add_label(&function_label);

    // 함수 프롤로그
    self.generate_function_prologue(function)?;

    // 기본 블록들 처리
    for basic_block in &function.basic_blocks {
        self.generate_basic_block_code(basic_block, type_info)?;
    }

    // 함수 에필로그
    self.generate_function_epilogue(function)?;

    Ok(())
}

fn generate_basic_block_code(
    &mut self,
    basic_block: &BasicBlock,
    type_info: &TypeInfo,
) -> Result<(), CompilationError> {
    for statement in &basic_block.statements {
        match statement {
            SierraStatement::Invocation(invocation) => {
                self.generate_invocation_code(invocation, type_info)?;
            }
            SierraStatement::Return(return_stmt) => {
                self.generate_return_code(return_stmt)?;
            }
            SierraStatement::Branch(branch) => {
```

```
                self.generate_branch_code(branch)?;
            }
        }
    }

    Ok(())
  }
}
```

7.2.4 최적화 시스템

CASM 최적화 패스:

```
#[derive(Debug, Clone, Copy)]
pub enum OptimizationLevel {
    None,       // 최적화 없음
    Basic,      // 기본 최적화
    Aggressive, // 적극적 최적화
}

impl SierraCompiler {
    fn optimize_casm(
        instructions: Vec<CasmInstruction>,
        config: &SierraCompilerConfig,
    ) -> Result<Vec<CasmInstruction>, CompilationError> {
        let mut optimized = instructions;

        match config.optimization_level {
            OptimizationLevel::None => {
                // 최적화 없음
            }
            OptimizationLevel::Basic => {
                // 기본 최적화 패스들
                optimized = Self::remove_dead_code(optimized)?;
                optimized = Self::fold_constants(optimized)?;
                optimized = Self::eliminate_redundant_moves(optimized)?;
            }
            OptimizationLevel::Aggressive => {
                // 기본 최적화 + 추가 최적화
                optimized = Self::remove_dead_code(optimized)?;
                optimized = Self::fold_constants(optimized)?;
                optimized = Self::eliminate_redundant_moves(optimized)?;
                optimized = Self::inline_small_functions(optimized)?;
                optimized = Self::optimize_loops(optimized)?;
                optimized = Self::reorder_instructions(optimized)?;
```

```
        }
    }

    Ok(optimized)
}

fn remove_dead_code(instructions: Vec<CasmInstruction>) -> Result<Vec<CasmIn
    let mut live_instructions = Vec::new();
    let mut used_registers = HashSet::new();

    // 역방향 분석으로 사용되는 레지스터 추적
    for instruction in instructions.iter().rev() {
        match instruction {
            CasmInstruction::AssertEq { a, b } => {
                used_registers.insert(a.clone());
                used_registers.insert(b.clone());
                live_instructions.push(instruction.clone());
            }
            CasmInstruction::Jump { target } => {
                live_instructions.push(instruction.clone());
            }
            CasmInstruction::Call { target } => {
                live_instructions.push(instruction.clone());
            }
            // 기타 명령어들...
            _ => {
                if Self::instruction_has_side_effects(instruction) ||
                   Self::instruction_defines_used_register(instruction, &use
                    live_instructions.push(instruction.clone());
                    Self::add_instruction_inputs_to_used_set(instruction, &m
                }
            }
        }
    }

    live_instructions.reverse();
    Ok(live_instructions)
}

fn fold_constants(instructions: Vec<CasmInstruction>) -> Result<Vec<CasmInst
    let mut folded_instructions = Vec::new();

    for instruction in instructions {
        match instruction {
            CasmInstruction::Add { a, b, dst } if Self::are_both_constants(&
                // 상수 덧셈 폴딩
```

```
                    let result = Self::add_constants(&a, &b)?;
                    folded_instructions.push(CasmInstruction::AssignImmediate {
                        dst,
                        value: result,
                    });
                }
                CasmInstruction::Mul { a, b, dst } if Self::are_both_constants(&
                    // 상수 곱셈 폴딩
                    let result = Self::multiply_constants(&a, &b)?;
                    folded_instructions.push(CasmInstruction::AssignImmediate {
                        dst,
                        value: result,
                    });
                }
                _ => {
                    folded_instructions.push(instruction);
                }
            }
        }

        Ok(folded_instructions)
    }
}
```

7.2.5 에러 처리 및 진단

컴파일 에러 타입:

```
#[derive(Debug, Error)]
pub enum CompilationError {
    #[error("Sierra parsing error: {message} at line {line}")]
    SierraParsingError { message: String, line: usize },

    #[error("Type checking error: {message}")]
    TypeCheckingError { message: String },

    #[error("Unsupported Sierra instruction: {instruction}")]
    UnsupportedInstruction { instruction: String },

    #[error("Code generation error: {message}")]
    CodeGenerationError { message: String },

    #[error("Optimization error: {message}")]
    OptimizationError { message: String },
```

```
    #[error("Thread panic during compilation: {message}")]
    ThreadPanic(String),

    #[error("Compilation timeout after {timeout_seconds} seconds")]
    CompilationTimeout { timeout_seconds: u64 },
}

impl SierraCompiler {
    fn validate_sierra_program(program: &ParsedSierraProgram) -> Result<(), Comp
        // 1. 구문 검증
        Self::validate_syntax(program)?;

        // 2. 타입 검증
        Self::validate_types(program)?;

        // 3. 제어 흐름 검증
        Self::validate_control_flow(program)?;

        // 4. 리소스 사용량 검증
        Self::validate_resource_usage(program)?;

        Ok(())
    }

    fn generate_compilation_diagnostics(
        &self,
        result: &CompilationResult,
    ) -> CompilationDiagnostics {
        CompilationDiagnostics {
            warnings: self.collect_compilation_warnings(result),
            performance_hints: self.generate_performance_hints(result),
            resource_usage: ResourceUsageReport {
                estimated_steps: self.estimate_execution_steps(&result.casm_clas
                memory_usage: self.estimate_memory_usage(&result.casm_class),
                builtin_usage: result.compilation_metadata.used_builtins.clone()
            },
        }
    }
}
```

8. 모니터링 및 메트릭스

8.1 Monitoring Endpoint (apollo_monitoring_endpoint)

역할: 시스템 상태를 실시간으로 모니터링하고 운영 정보를 제공하는 중앙 집중식 모니터링 서비스

8.1.1 Monitoring Endpoint 구조

```rust
pub struct MonitoringEndpoint {
    config: MonitoringEndpointConfig,
    // 버전 정보
    version_info: &'static str,
    // 외부 서비스 클라이언트들
    mempool_client: Option<SharedMempoolClient>,
    l1_provider_client: Option<SharedL1ProviderClient>,
    // 시스템 메트릭 수집기
    system_metrics_collector: SystemMetricsCollector,
    // 헬스 체크 상태
    health_status: Arc<RwLock<HealthStatus>>,
    // 모니터링 통계
    monitoring_stats: MonitoringStats,
}

#[derive(Debug, Clone)]
pub struct HealthStatus {
    // 전체 시스템 상태
    overall_status: ServiceStatus,
    // 개별 컴포넌트 상태
    component_statuses: HashMap<String, ComponentHealth>,
    // 마지막 업데이트 시간
    last_updated: DateTime<Utc>,
    // 시스템 가동 시간
    uptime: Duration,
}

#[derive(Debug, Clone)]
pub enum ServiceStatus {
    Healthy,
    Degraded { reason: String },
    Unhealthy { reason: String },
    Unknown,
}

#[derive(Debug, Clone)]
pub struct ComponentHealth {
    status: ServiceStatus,
```

```
    last_check: DateTime<Utc>,
    response_time: Option<Duration>,
    error_count: u64,
    success_rate: f64,
}
```

8.1.2 헬스 체크 시스템

종합 헬스 체크:

```
impl MonitoringEndpoint {
    pub async fn perform_health_check(&self) -> HealthCheckResult {
        let start_time = Instant::now();
        let mut component_results = HashMap::new();

        // 1. 개별 컴포넌트 헬스 체크
        let health_checks = vec![
            self.check_mempool_health(),
            self.check_l1_provider_health(),
            self.check_database_health(),
            self.check_network_connectivity(),
            self.check_system_resources(),
        ];

        let results = futures::future::join_all(health_checks).await;

        // 2. 결과 수집 및 분석
        for (component_name, result) in results.into_iter().enumerate() {
            let component_name = match component_name {
                0 => "mempool",
                1 => "l1_provider",
                2 => "database",
                3 => "network",
                4 => "system",
                _ => "unknown",
            };

            component_results.insert(component_name.to_string(), result);
        }

        // 3. 전체 상태 결정
        let overall_status = self.determine_overall_status(&component_results);

        // 4. 헬스 상태 업데이트
        let health_status = HealthStatus {
```

```
        overall_status,
        component_statuses: component_results.clone(),
        last_updated: Utc::now(),
        uptime: start_time.elapsed(),
    };

    self.update_health_status(health_status).await;

    HealthCheckResult {
        status: overall_status,
        components: component_results,
        check_duration: start_time.elapsed(),
        timestamp: Utc::now(),
    }
}

async fn check_mempool_health(&self) -> ComponentHealth {
    if let Some(mempool_client) = &self.mempool_client {
        let start_time = Instant::now();

        match mempool_client.get_mempool_snapshot().await {
            Ok(snapshot) => {
                let response_time = start_time.elapsed();

                // 메모리 풀 상태 분석
                let status = if snapshot.transactions.len() > self.config.me
                    ServiceStatus::Degraded {
                        reason: format!("High transaction count: {}", snapsh
                    }
                } else {
                    ServiceStatus::Healthy
                };

                ComponentHealth {
                    status,
                    last_check: Utc::now(),
                    response_time: Some(response_time),
                    error_count: 0,
                    success_rate: 100.0,
                }
            }
            Err(err) => {
                ComponentHealth {
                    status: ServiceStatus::Unhealthy {
                        reason: format!("Mempool check failed: {}", err)
                    },
```

```
                    last_check: Utc::now(),
                    response_time: None,
                    error_count: 1,
                    success_rate: 0.0,
                }
            }
        }
    } else {
        ComponentHealth {
            status: ServiceStatus::Unknown,
            last_check: Utc::now(),
            response_time: None,
            error_count: 0,
            success_rate: 0.0,
        }
    }
}

async fn check_system_resources(&self) -> ComponentHealth {
    let system_info = self.system_metrics_collector.collect_system_info().aw

    let mut issues = Vec::new();

    // CPU 사용률 체크
    if system_info.cpu_usage > self.config.cpu_warning_threshold {
        issues.push(format!("High CPU usage: {:.1}%", system_info.cpu_usage)
    }

    // 메모리 사용률 체크
    if system_info.memory_usage > self.config.memory_warning_threshold {
        issues.push(format!("High memory usage: {:.1}%", system_info.memory_
    }

    // 디스크 사용률 체크
    if system_info.disk_usage > self.config.disk_warning_threshold {
        issues.push(format!("High disk usage: {:.1}%", system_info.disk_usag
    }

    let status = if issues.is_empty() {
        ServiceStatus::Healthy
    } else if issues.len() == 1 {
        ServiceStatus::Degraded { reason: issues[0].clone() }
    } else {
        ServiceStatus::Unhealthy { reason: issues.join(", ") }
    };
```

```
        ComponentHealth {
            status,
            last_check: Utc::now(),
            response_time: Some(Duration::from_millis(10)), // 시스템 정보 수집은
            error_count: 0,
            success_rate: 100.0,
        }
    }
}
```

8.1.3 시스템 메트릭 수집

시스템 정보 수집기:

```
pub struct SystemMetricsCollector {
    // 시스템 정보 캐시
    cached_info: Arc<RwLock<Option<(SystemInfo, Instant)>>>,
    cache_duration: Duration,
}

#[derive(Debug, Clone)]
pub struct SystemInfo {
    // CPU 정보
    cpu_usage: f64,
    cpu_cores: usize,
    load_average: [f64; 3], // 1분, 5분, 15분

    // 메모리 정보
    memory_total: u64,
    memory_used: u64,
    memory_usage: f64,

    // 디스크 정보
    disk_total: u64,
    disk_used: u64,
    disk_usage: f64,

    // 네트워크 정보
    network_rx_bytes: u64,
    network_tx_bytes: u64,
    network_rx_packets: u64,
    network_tx_packets: u64,

    // 프로세스 정보
```

```rust
    process_count: usize,
    thread_count: usize,
    file_descriptor_count: usize,

    // 시스템 가동 시간
    uptime: Duration,

    // 수집 시간
    collected_at: DateTime<Utc>,
}

impl SystemMetricsCollector {
    pub async fn collect_system_info(&self) -> SystemInfo {
        // 캐시된 정보 확인
        {
            let cached = self.cached_info.read().await;
            if let Some((info, cached_at)) = cached.as_ref() {
                if cached_at.elapsed() < self.cache_duration {
                    return info.clone();
                }
            }
        }

        // 새로운 시스템 정보 수집
        let system_info = self.collect_fresh_system_info().await;

        // 캐시 업데이트
        {
            let mut cached = self.cached_info.write().await;
            *cached = Some((system_info.clone(), Instant::now()));
        }

        system_info
    }

    async fn collect_fresh_system_info(&self) -> SystemInfo {
        // CPU 정보 수집
        let cpu_usage = self.get_cpu_usage().await;
        let cpu_cores = num_cpus::get();
        let load_average = self.get_load_average().await;

        // 메모리 정보 수집
        let (memory_total, memory_used) = self.get_memory_info().await;
        let memory_usage = (memory_used as f64 / memory_total as f64) * 100.0;

        // 디스크 정보 수집
```

```
        let (disk_total, disk_used) = self.get_disk_info().await;
        let disk_usage = (disk_used as f64 / disk_total as f64) * 100.0;

        // 네트워크 정보 수집
        let network_stats = self.get_network_stats().await;

        // 프로세스 정보 수집
        let process_info = self.get_process_info().await;

        // 시스템 가동 시간
        let uptime = self.get_system_uptime().await;

        SystemInfo {
            cpu_usage,
            cpu_cores,
            load_average,
            memory_total,
            memory_used,
            memory_usage,
            disk_total,
            disk_used,
            disk_usage,
            network_rx_bytes: network_stats.rx_bytes,
            network_tx_bytes: network_stats.tx_bytes,
            network_rx_packets: network_stats.rx_packets,
            network_tx_packets: network_stats.tx_packets,
            process_count: process_info.process_count,
            thread_count: process_info.thread_count,
            file_descriptor_count: process_info.fd_count,
            uptime,
            collected_at: Utc::now(),
        }
    }
}
```

8.1.4 모니터링 API 엔드포인트

REST API 제공:

```
impl MonitoringEndpoint {
    pub fn create_monitoring_routes(&self) -> Router {
        Router::new()
            .route("/health", get(self.handle_health_check()))
            .route("/metrics", get(self.handle_metrics_request()))
            .route("/status", get(self.handle_status_request()))
```

```
        .route("/version", get(self.handle_version_request()))
        .route("/system", get(self.handle_system_info_request()))
}

async fn handle_health_check(&self) -> Result<Json<HealthCheckResponse>, Sta
    let health_result = self.perform_health_check().await;

    let response = HealthCheckResponse {
        status: health_result.status.clone(),
        timestamp: health_result.timestamp,
        uptime_seconds: health_result.check_duration.as_secs(),
        components: health_result.components,
        version: self.version_info,
    };

    let status_code = match health_result.status {
        ServiceStatus::Healthy => StatusCode::OK,
        ServiceStatus::Degraded { .. } => StatusCode::OK, // 여전히 서비스 가능
        ServiceStatus::Unhealthy { .. } => StatusCode::SERVICE_UNAVAILABLE,
        ServiceStatus::Unknown => StatusCode::SERVICE_UNAVAILABLE,
    };

    Ok((status_code, Json(response)).into_response())
}

async fn handle_metrics_request(&self) -> Result<String, StatusCode> {
    // Prometheus 형식의 메트릭 생성
    let mut metrics_output = String::new();

    // 시스템 메트릭
    let system_info = self.system_metrics_collector.collect_system_info().aw
    metrics_output.push_str(&format!(
        "# HELP system_cpu_usage CPU usage percentage\n\
         # TYPE system_cpu_usage gauge\n\
         system_cpu_usage {}\n\n",
        system_info.cpu_usage
    ));

    metrics_output.push_str(&format!(
        "# HELP system_memory_usage Memory usage percentage\n\
         # TYPE system_memory_usage gauge\n\
         system_memory_usage {}\n\n",
        system_info.memory_usage
    ));

    // 컴포넌트별 메트릭
```

```
        let health_status = self.health_status.read().await;
        for (component_name, component_health) in &health_status.component_statu
            let status_value = match component_health.status {
                ServiceStatus::Healthy => 1.0,
                ServiceStatus::Degraded { .. } => 0.5,
                ServiceStatus::Unhealthy { .. } => 0.0,
                ServiceStatus::Unknown => -1.0,
            };

            metrics_output.push_str(&format!(
                "# HELP component_health Component health status\n\
                 # TYPE component_health gauge\n\
                 component_health{{component=\"{}\"}} {}\n\n",
                component_name, status_value
            ));
        }

        Ok(metrics_output)
    }

    async fn handle_status_request(&self) -> Json<NodeStatusResponse> {
        let health_status = self.health_status.read().await;
        let system_info = self.system_metrics_collector.collect_system_info().aw

        Json(NodeStatusResponse {
            node_id: self.config.node_id.clone(),
            version: self.version_info,
            status: health_status.overall_status.clone(),
            uptime_seconds: health_status.uptime.as_secs(),
            system_info: system_info.clone(),
            component_count: health_status.component_statuses.len(),
            last_updated: health_status.last_updated,
        })
    }
}
```

8.2 Metrics Collection (apollo_metrics)

역할: 시스템 전반의 성능 메트릭을 수집, 집계, 저장하는 중앙 집중식 메트릭 시스템

8.2.1 메트릭 시스템 구조

```rust
pub struct MetricsRegistry {
    // 메트릭 저장소
    metrics: Arc<RwLock<HashMap<String, MetricValue>>>,
    // 메트릭 메타데이터
    metadata: Arc<RwLock<HashMap<String, MetricMetadata>>>,
    // 집계 규칙
    aggregation_rules: Vec<AggregationRule>,
    // 내보내기 설정
    exporters: Vec<Box<dyn MetricExporter>>,
}

#[derive(Debug, Clone)]
pub enum MetricValue {
    Counter(u64),
    Gauge(f64),
    Histogram(HistogramData),
    Summary(SummaryData),
}

#[derive(Debug, Clone)]
pub struct MetricMetadata {
    name: String,
    description: String,
    unit: Option<String>,
    labels: HashMap<String, String>,
    metric_type: MetricType,
    created_at: DateTime<Utc>,
}

#[derive(Debug, Clone)]
pub enum MetricType {
    Counter,
    Gauge,
    Histogram,
    Summary,
}
```

8.2.2 핵심 메트릭 정의

시스템 전반 메트릭:

```rust
// 트랜잭션 관련 메트릭
lazy_static! {
    pub static ref TRANSACTION_RECEIVED_TOTAL: Counter = Counter::new(
```

```rust
    "transaction_received_total",
    "Total number of transactions received"
);

pub static ref TRANSACTION_PROCESSED_TOTAL: Counter = Counter::new(
    "transaction_processed_total",
    "Total number of transactions processed"
);

pub static ref TRANSACTION_PROCESSING_DURATION: Histogram = Histogram::with_
    HistogramOpts::new(
        "transaction_processing_duration_seconds",
        "Time spent processing transactions"
    ).buckets(vec![0.001, 0.01, 0.1, 1.0, 10.0])
);

// 블록 관련 메트릭
pub static ref BLOCK_CREATED_TOTAL: Counter = Counter::new(
    "block_created_total",
    "Total number of blocks created"
);

pub static ref BLOCK_CREATION_DURATION: Histogram = Histogram::with_opts(
    HistogramOpts::new(
        "block_creation_duration_seconds",
        "Time spent creating blocks"
    ).buckets(vec![0.1, 0.5, 1.0, 5.0, 10.0, 30.0])
);

pub static ref BLOCK_SIZE_BYTES: Histogram = Histogram::with_opts(
    HistogramOpts::new(
        "block_size_bytes",
        "Size of created blocks in bytes"
    ).buckets(exponential_buckets(1024.0, 2.0, 20))
);

// 메모리 풀 메트릭
pub static ref MEMPOOL_SIZE: Gauge = Gauge::new(
    "mempool_size",
    "Current number of transactions in mempool"
);

pub static ref MEMPOOL_SIZE_BYTES: Gauge = Gauge::new(
    "mempool_size_bytes",
    "Current size of mempool in bytes"
);
```

```rust
    // 네트워크 메트릭
    pub static ref NETWORK_PEERS_CONNECTED: Gauge = Gauge::new(
        "network_peers_connected",
        "Number of connected peers"
    );

    pub static ref NETWORK_MESSAGES_SENT_TOTAL: Counter = Counter::new(
        "network_messages_sent_total",
        "Total number of network messages sent"
    );

    pub static ref NETWORK_MESSAGES_RECEIVED_TOTAL: Counter = Counter::new(
        "network_messages_received_total",
        "Total number of network messages received"
    );
}
```

8.2.3 메트릭 수집 및 집계

자동 메트릭 수집:

```rust
pub struct MetricsCollector {
    registry: Arc<MetricsRegistry>,
    collection_interval: Duration,
    collectors: Vec<Box<dyn ComponentMetricsCollector>>,
}

#[async_trait]
pub trait ComponentMetricsCollector: Send + Sync {
    async fn collect_metrics(&self) -> Result<Vec<MetricSample>, MetricsError>;
    fn component_name(&self) -> &str;
}

pub struct MempoolMetricsCollector {
    mempool_client: SharedMempoolClient,
}

#[async_trait]
impl ComponentMetricsCollector for MempoolMetricsCollector {
    async fn collect_metrics(&self) -> Result<Vec<MetricSample>, MetricsError> {
        let snapshot = self.mempool_client.get_mempool_snapshot().await?;

        let mut metrics = Vec::new();
```

```
        // 메모리 풀 크기
        metrics.push(MetricSample {
            name: "mempool_transaction_count".to_string(),
            value: MetricValue::Gauge(snapshot.transactions.len() as f64),
            labels: HashMap::new(),
            timestamp: Utc::now(),
        });

        // 지연된 선언 트랜잭션 수
        metrics.push(MetricSample {
            name: "mempool_delayed_declares_count".to_string(),
            value: MetricValue::Gauge(snapshot.delayed_declares.len() as f64),
            labels: HashMap::new(),
            timestamp: Utc::now(),
        });

        // 우선순위 큐 크기
        metrics.push(MetricSample {
            name: "mempool_priority_queue_size".to_string(),
            value: MetricValue::Gauge(snapshot.transaction_queue.len() as f64),
            labels: HashMap::new(),
            timestamp: Utc::now(),
        });

        Ok(metrics)
    }

    fn component_name(&self) -> &str {
        "mempool"
    }
}

impl MetricsCollector {
    pub async fn start_collection(&mut self) {
        let mut interval = tokio::time::interval(self.collection_interval);

        loop {
            interval.tick().await;

            // 모든 컴포넌트에서 메트릭 수집
            for collector in &self.collectors {
                match collector.collect_metrics().await {
                    Ok(metrics) => {
                        for metric in metrics {
                            self.registry.record_metric(metric).await;
                        }
```

```
                }
                Err(err) => {
                    warn!("Failed to collect metrics from {}: {}",
                          collector.component_name(), err);
                }
            }
        }

        // 집계 규칙 적용
        self.apply_aggregation_rules().await;

        // 메트릭 내보내기
        self.export_metrics().await;
    }
}

async fn apply_aggregation_rules(&self) {
    for rule in &self.registry.aggregation_rules {
        match rule.rule_type {
            AggregationRuleType::Sum => {
                self.apply_sum_aggregation(rule).await;
            }
            AggregationRuleType::Average => {
                self.apply_average_aggregation(rule).await;
            }
            AggregationRuleType::Rate => {
                self.apply_rate_calculation(rule).await;
            }
        }
    }
}
}
```

8.2.4 성능 메트릭 분석

처리량 및 지연시간 추적:

```
pub struct PerformanceAnalyzer {
    // 시계열 데이터 저장소
    time_series_store: Arc<TimeSeriesStore>,
    // 분석 윈도우 크기
    analysis_window: Duration,
    // 알림 임계값
    alert_thresholds: HashMap<String, AlertThreshold>,
}
```

```rust
#[derive(Debug, Clone)]
pub struct PerformanceMetrics {
    // 트랜잭션 처리량 (TPS)
    transaction_throughput: f64,
    // 평균 트랜잭션 처리 시간
    avg_transaction_latency: Duration,
    // 95퍼센타일 지연시간
    p95_transaction_latency: Duration,
    // 99퍼센타일 지연시간
    p99_transaction_latency: Duration,
    // 블록 생성 간격
    avg_block_interval: Duration,
    // 메모리 풀 처리 효율성
    mempool_efficiency: f64,
    // 네트워크 처리량
    network_throughput_mbps: f64,
    // 시스템 리소스 사용률
    resource_utilization: ResourceUtilization,
}

impl PerformanceAnalyzer {
    pub async fn analyze_performance(&self, time_range: TimeRange) -> Performanc
        // 1. 시계열 데이터 조회
        let transaction_data = self.time_series_store
            .query_metric("transaction_processed_total", time_range)
            .await;
        let latency_data = self.time_series_store
            .query_metric("transaction_processing_duration", time_range)
            .await;

        // 2. 처리량 계산
        let throughput = self.calculate_throughput(&transaction_data);

        // 3. 지연시간 분석
        let latency_stats = self.analyze_latency_distribution(&latency_data);

        // 4. 트렌드 분석
        let trends = self.analyze_trends(&transaction_data, &latency_data);

        // 5. 이상 탐지
        let anomalies = self.detect_anomalies(&transaction_data, &latency_data);

        // 6. 성능 점수 계산
        let performance_score = self.calculate_performance_score(&latency_stats,
```

```
        PerformanceReport {
            time_range,
            throughput,
            latency_statistics: latency_stats,
            trends,
            anomalies,
            performance_score,
            recommendations: self.generate_recommendations(&latency_stats, throu
        }
    }

    fn calculate_throughput(&self, transaction_data: &[TimeSeriesPoint]) -> f64
        if transaction_data.len() < 2 {
            return 0.0;
        }

        let start_point = &transaction_data[0];
        let end_point = &transaction_data[transaction_data.len() - 1];

        let transaction_count = end_point.value - start_point.value;
        let time_diff = end_point.timestamp - start_point.timestamp;

        transaction_count / time_diff.as_secs_f64()
    }

    fn detect_anomalies(
        &self,
        transaction_data: &[TimeSeriesPoint],
        latency_data: &[TimeSeriesPoint],
    ) -> Vec<Anomaly> {
        let mut anomalies = Vec::new();

        // 처리량 이상 탐지
        let throughput_anomalies = self.detect_throughput_anomalies(transaction_
        anomalies.extend(throughput_anomalies);

        // 지연시간 이상 탐지
        let latency_anomalies = self.detect_latency_anomalies(latency_data);
        anomalies.extend(latency_anomalies);

        anomalies
    }
}
```

8.2.5 알림 및 경고 시스템

임계값 기반 알림:

```rust
pub struct AlertManager {
    // 알림 규칙
    alert_rules: Vec<AlertRule>,
    // 알림 채널
    notification_channels: Vec<Box<dyn NotificationChannel>>,
    // 활성 알림
    active_alerts: Arc<RwLock<HashMap<String, ActiveAlert>>>,
    // 알림 히스토리
    alert_history: Arc<Mutex<VecDeque<AlertEvent>>>,
}

#[derive(Debug, Clone)]
pub struct AlertRule {
    name: String,
    description: String,
    metric_name: String,
    condition: AlertCondition,
    severity: AlertSeverity,
    duration: Duration, // 조건이 지속되어야 하는 시간
    cooldown: Duration, // 알림 간 최소 간격
}

#[derive(Debug, Clone)]
pub enum AlertCondition {
    GreaterThan(f64),
    LessThan(f64),
    Equal(f64),
    PercentageChange { threshold: f64, window: Duration },
    RateOfChange { threshold: f64, window: Duration },
}

#[derive(Debug, Clone)]
pub enum AlertSeverity {
    Info,
    Warning,
    Critical,
    Emergency,
}

impl AlertManager {
    pub async fn evaluate_alerts(&mut self) {
        for rule in &self.alert_rules {
            match self.evaluate_alert_rule(rule).await {
```

```
            Ok(should_alert) => {
                if should_alert {
                    self.trigger_alert(rule).await;
                } else {
                    self.resolve_alert(rule).await;
                }
            }
            Err(err) => {
                warn!("Failed to evaluate alert rule {}: {}", rule.name, err
            }
        }
    }
}

async fn evaluate_alert_rule(&self, rule: &AlertRule) -> Result<bool, AlertE
    // 메트릭 값 조회
    let current_value = self.get_current_metric_value(&rule.metric_name).awa

    // 조건 평가
    let condition_met = match &rule.condition {
        AlertCondition::GreaterThan(threshold) => current_value > *threshold
        AlertCondition::LessThan(threshold) => current_value < *threshold,
        AlertCondition::Equal(threshold) => (current_value - threshold).abs(
        AlertCondition::PercentageChange { threshold, window } => {
            let historical_value = self.get_historical_metric_value(
                &rule.metric_name,
                *window
            ).await?;
            let percentage_change = ((current_value - historical_value) / hi
            percentage_change.abs() > *threshold
        }
        AlertCondition::RateOfChange { threshold, window } => {
            let rate = self.calculate_metric_rate(&rule.metric_name, *window
            rate > *threshold
        }
    };

    if condition_met {
        // 지속 시간 확인
        self.check_condition_duration(rule).await
    } else {
        Ok(false)
    }
}

async fn trigger_alert(&mut self, rule: &AlertRule) {
```

```rust
        let alert_id = format!("{}_{}", rule.name, Utc::now().timestamp());

        let alert = ActiveAlert {
            id: alert_id.clone(),
            rule_name: rule.name.clone(),
            severity: rule.severity.clone(),
            message: format!("Alert triggered: {}", rule.description),
            triggered_at: Utc::now(),
            acknowledged: false,
        };

        // 활성 알림에 추가
        self.active_alerts.write().await.insert(alert_id.clone(), alert.clone())

        // 알림 전송
        for channel in &self.notification_channels {
            if let Err(err) = channel.send_alert(&alert).await {
                warn!("Failed to send alert via {}: {}", channel.name(), err);
            }
        }

        // 히스토리에 기록
        let alert_event = AlertEvent {
            alert_id,
            event_type: AlertEventType::Triggered,
            timestamp: Utc::now(),
            details: serde_json::to_value(&alert).unwrap_or_default(),
        };

        self.alert_history.lock().await.push_back(alert_event);
    }
}

#[async_trait]
pub trait NotificationChannel: Send + Sync {
    async fn send_alert(&self, alert: &ActiveAlert) -> Result<(), NotificationEr
    fn name(&self) -> &str;
}

pub struct SlackNotificationChannel {
    webhook_url: String,
    channel: String,
}

#[async_trait]
impl NotificationChannel for SlackNotificationChannel {
```

```
    async fn send_alert(&self, alert: &ActiveAlert) -> Result<(), NotificationEr
        let payload = json!({
            "channel": self.channel,
            "text": format!("🚨 {} Alert", alert.severity),
            "attachments": [{
                "color": match alert.severity {
                    AlertSeverity::Info => "good",
                    AlertSeverity::Warning => "warning",
                    AlertSeverity::Critical => "danger",
                    AlertSeverity::Emergency => "danger",
                },
                "fields": [
                    {
                        "title": "Alert",
                        "value": alert.rule_name,
                        "short": true
                    },
                    {
                        "title": "Message",
                        "value": alert.message,
                        "short": false
                    },
                    {
                        "title": "Time",
                        "value": alert.triggered_at.to_rfc3339(),
                        "short": true
                    }
                ]
            }]
        });

        let client = reqwest::Client::new();
        let response = client
            .post(&self.webhook_url)
            .json(&payload)
            .send()
            .await?;

        if response.status().is_success() {
            Ok(())
        } else {
            Err(NotificationError::SendFailed(format!(
                "Slack API returned status: {}",
                response.status()
            )))
        }
```

```
    }

    fn name(&self) -> &str {
        "slack"
    }
}
```

9. 설정 및 배포

9.1 Configuration (apollo_config)

설정 관리:

- 컴포넌트별 설정
- 네트워크 설정
- 보안 설정
- 성능 튜닝 파라미터

9.2 Deployment

배포 구조:

```
deployments/
├── sequencer/          # 시퀀서 배포 설정
├── papyrus/           # Papyrus 노드 설정
├── monitoring/        # 모니터링 설정
└── images/           # Docker 이미지
```

10. 테스팅 및 품질 보증

10.1 테스트 구조

- **Unit Tests**: 개별 컴포넌트 테스트
- **Integration Tests**: 컴포넌트 간 통합 테스트
- **Load Tests**: 성능 및 부하 테스트
- **Flow Tests**: 전체 워크플로우 테스트

10.2 품질 관리

- **Linting**: Clippy를 통한 코드 품질 검사
- **Formatting**: rustfmt를 통한 코드 포맷팅
- **CI/CD**: 자동화된 테스트 및 배포

11. 성능 최적화

11.1 동시성 처리

- **Async/Await**: 비동기 처리를 통한 성능 향상
- **Tokio Runtime**: 고성능 비동기 런타임
- **Concurrent Execution**: 병렬 트랜잭션 실행

11.2 메모리 관리

- **JeMalloc**: 고성능 메모리 할당자
- **Zero-Copy**: 불필요한 복사 최소화
- **Memory Pool**: 메모리 풀을 통한 효율적 관리

12. 보안 고려사항

12.1 트랜잭션 검증

- **Signature Verification**: 디지털 서명 검증
- **Nonce Validation**: 중복 실행 방지
- **Gas Limit Checks**: 가스 한도 검증

12.2 네트워크 보안

- **Peer Authentication**: 피어 인증
- **Message Validation**: 메시지 무결성 검증
- **DDoS Protection**: 서비스 거부 공격 방어

13. 결론

13.1 기술적 성과

이 Starknet 시퀀서 구현체는 현대적인 Rust 생태계를 활용하여 고성능, 확장 가능한 L2 블록체인 인프라를 제공합니다. **Apollo**라는 코드명으로 개발된 이 시스템은 다음과 같은 핵심 기술적 성과를 달성했습니다:

13.1.1 아키텍처 혁신

모듈화된 마이크로서비스 아키텍처:

- 18개의 핵심 컴포넌트로 구성된 느슨하게 결합된 시스템
- 각 컴포넌트의 독립적인 개발, 테스트, 배포 가능
- 장애 격리 및 부분적 업그레이드 지원
- 수평적 확장성을 고려한 설계

비동기 이벤트 기반 처리:

- Tokio 런타임 기반의 완전 비동기 처리
- 논블로킹 I/O를 통한 높은 동시성 달성
- 백프레셔 메커니즘을 통한 시스템 안정성 보장

13.1.2 성능 최적화

메모리 관리 최적화:

- JeMalloc 사용으로 메모리 할당 성능 향상
- LRU 캐시를 통한 효율적인 데이터 캐싱
- Zero-copy 기법으로 불필요한 메모리 복사 최소화

병렬 처리 및 동시성:

- 워커 풀 기반 병렬 트랜잭션 실행
- 충돌 감지 및 해결을 통한 안전한 병렬 처리
- 락-프리 데이터 구조 활용으로 경합 최소화

네트워크 최적화:

- libp2p 기반 효율적인 P2P 통신

- 메시지 압축 및 배치 전송
- 적응적 대역폭 관리

13.1.3 확장성 및 신뢰성

수평적 확장성:

- 상태 없는 컴포넌트 설계로 쉬운 스케일 아웃
- 로드 밸런싱 및 샤딩 지원
- 클라우드 네이티브 배포 최적화

장애 복구 메커니즘:

- 체크포인트 기반 상태 롤백
- 자동 재시도 및 회로 차단기 패턴
- 그레이스풀 셧다운 및 재시작

데이터 일관성:

- ACID 트랜잭션 보장
- 분산 합의를 통한 데이터 일관성
- 암호학적 검증을 통한 무결성 보장

13.2 보안 및 검증

13.2.1 다층 보안 아키텍처

트랜잭션 레벨 보안:

- ECDSA 서명 검증
- Nonce 기반 재생 공격 방지
- 가스 한도를 통한 DoS 공격 방어

네트워크 레벨 보안:

- 피어 인증 및 평판 시스템
- 메시지 무결성 검증
- DDoS 방어 메커니즘

시스템 레벨 보안:

- 메모리 안전성 (Rust의 소유권 시스템)
- 타입 안전성을 통한 런타임 에러 방지
- 보안 감사 및 정적 분석

13.2.2 암호학적 검증

상태 증명:

- Patricia Tree 기반 머클 증명
- zk-STARK 증명 시스템 통합
- L1 상태 루트 검증

트랜잭션 검증:

- Cairo VM을 통한 실행 검증
- 리소스 사용량 추적 및 제한
- 실행 결과 일관성 검증

13.3 운영 및 모니터링

13.3.1 관찰 가능성 (Observability)

메트릭 수집:

- Prometheus 호환 메트릭 내보내기
- 실시간 성능 지표 추적
- 비즈니스 메트릭 및 SLA 모니터링

로깅 및 추적:

- 구조화된 로깅 (JSON 형식)
- 분산 추적 (OpenTelemetry 호환)
- 에러 추적 및 알림

헬스 체크:

- 다층 헬스 체크 시스템
- 자동 장애 감지 및 복구
- 서비스 디스커버리 통합

13.3.2 운영 도구

배포 및 관리:

- Docker 컨테이너 기반 배포
- Kubernetes 오케스트레이션 지원
- 무중단 업그레이드 지원

디버깅 및 진단:

- 실시간 시스템 상태 조회
- 성능 프로파일링 도구
- 메모리 및 CPU 사용량 분석

13.4 생태계 통합

13.4.1 Starknet 생태계

Cairo 언어 지원:

- Cairo 0.x 및 1.x 완전 지원
- Sierra 중간 표현 최적화
- CASM 바이트코드 생성 및 실행

개발자 도구:

- RPC API 완전 호환
- 개발자 친화적 에러 메시지
- 테스트넷 및 메인넷 지원

13.4.2 Ethereum 통합

L1-L2 브릿지:

- 양방향 메시지 전달
- 자산 브릿징 지원
- 최종성 보장 메커니즘

상태 증명:

- 주기적 상태 루트 제출
- 사기 증명 시스템
- 검증자 네트워크 지원

13.5 미래 발전 방향

13.5.1 성능 개선

처리량 향상:

- 병렬 실행 최적화
- 배치 처리 개선
- 네트워크 프로토콜 최적화

지연시간 감소:

- 사전 실행 (Pre-execution)
- 투기적 실행 (Speculative Execution)
- 캐싱 전략 개선

13.5.2 기능 확장

새로운 기능:

- 계정 추상화 (Account Abstraction)
- 볼륨 기반 수수료 모델
- 크로스 체인 상호 운용성

개발자 경험:

- 향상된 디버깅 도구
- 성능 분석 도구
- 자동화된 테스트 프레임워크

13.6 최종 평가

이 Starknet 시퀀서 구현체는 다음과 같은 핵심 가치를 제공합니다:

기술적 우수성:

- 현대적인 시스템 설계 원칙 적용
- 고성능 및 확장성 달성
- 강력한 보안 및 신뢰성 보장

운영 효율성:

- 자동화된 운영 및 모니터링
- 효율적인 리소스 사용
- 간편한 배포 및 관리

개발자 친화성:

- 명확한 API 및 문서
- 풍부한 개발 도구
- 활발한 커뮤니티 지원

생태계 기여:

- Starknet 네트워크 성장 촉진
- 오픈소스 기여 및 협업
- 블록체인 기술 발전에 기여

이러한 종합적인 접근을 통해 Starknet 시퀀서는 차세대 L2 블록체인 인프라의 표준을 제시하며, 확장 가능하고 안전한 분산 애플리케이션 생태계의 기반을 제공합니다. 지속적인 개선과 커뮤니티 피드백을 통해 더욱 발전된 시스템으로 진화할 것으로 기대됩니다.